Universidade Estadual da Paraíba

Centro de Ciências e Tecnologia - CCT

Departamento de Computação

Curso de Ciência da Computação

Disciplina: Compiladores

Análise Sintática

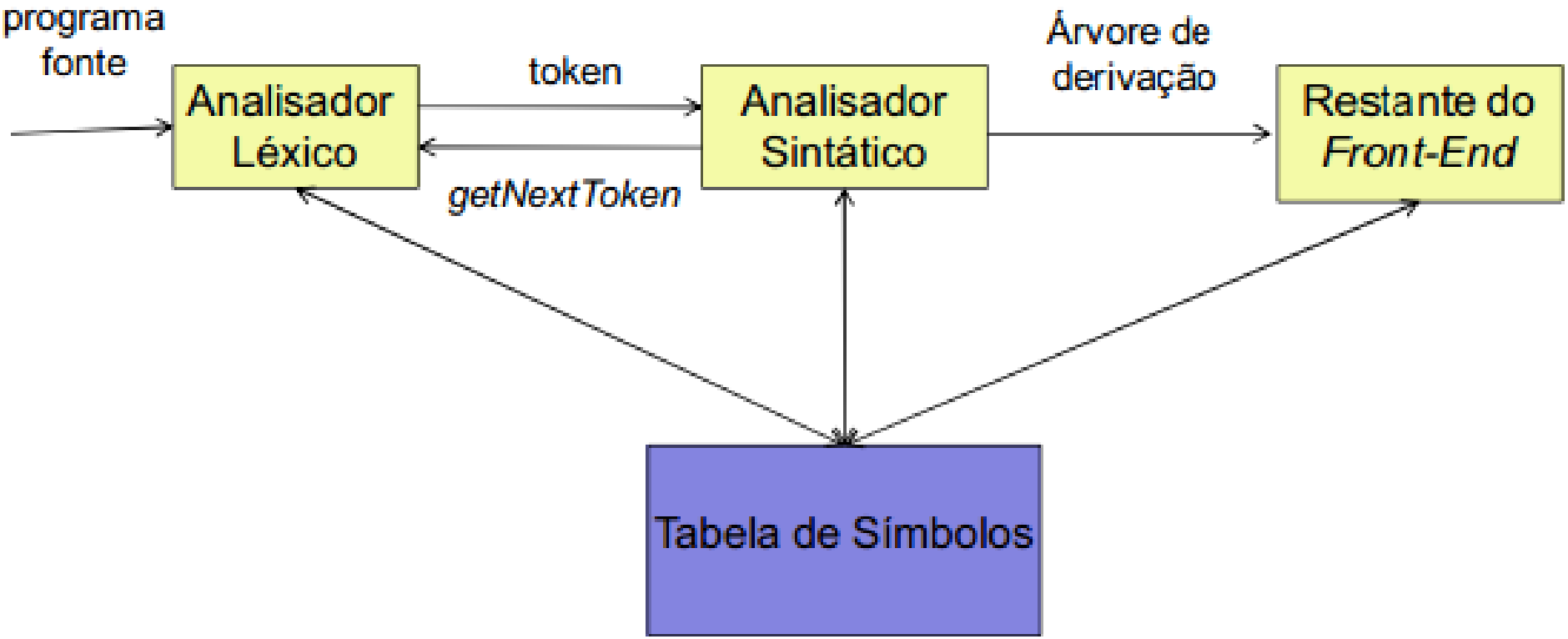
Professora: Kézia de Vasconcelos Oliveira Dantas

kezia.vasconcelos@gmail.com

Analisador Sintático

- Recebe uma sequência de tokens do analisador léxico e determina se a string pode ser gerada através da gramática da linguagem fonte
- Se o programa estiver bem formado, o analisador constrói uma árvore de derivação
- É esperado que ele reporte os erros de uma maneira inteligível
- Deve se recuperar de erros comuns, continuando a processar a entrada

Papel do Analisador Sintático



O Papel do Analisador Sintático

- Na prática existem diversas tarefas que poderiam ser conduzidas durante a análise sintática:
  - Coletar informações sobre os tokens na tabela de símbolos
  - Verificação de tipos
  - Geração de código intermediário
  - Todas essas atividades estão reunidas na caixa "Restante do Front-End"

Sintaxe de uma Linguagem

- Define as construções estruturalmente válidas de uma linguagem

- "Fez parsing corretamente" = A sintaxe está correta
  - Observação: O programa pode ainda...
    - não passar na checagem de tipos
    - ou (de forma mais geral) conter erros

Sintaxe de uma Linguagem de Programação

- ERs são boas para definir a estrutura léxica de maneira declarativa

- Não são "poderosas" o suficiente para conseguir definir declarativamente a estrutura sintática de linguagens de programação

Sintaxe de uma Linguagem de Programação

- Exemplo de ER usando abreviações:
  - digits = $[0\text{-}9]^+$
  - sum = (digits "+")* digits
  - definem somas da forma 28+301+9

- Como isso é implementado?
  - O analisador léxico substitui as abreviações antes de traduzir para um autômato finito
  - sum = ($[0\text{-}9]^+$ "+") * $[0\text{-}9]^+$

Sintaxe de uma Linguagem de Programação

- É possível usar a mesma ideia para definir uma linguagem para expressões que tenham parênteses balanceados?
  - (1+(245+2))

- Tentativa:
  - digits = $[0\text{-}9]^+$
  - sum = expr "+" expr
  - expr = "(" sum ")" | digits

Sintaxe de uma Linguagem de Programação

- O analisador léxico substituiria *sum em expr:*
  - expr = “(“ expr “+” expr “)” | digits
- Depois substituiria expr no próprio expr:
  - expr = “(“ (“(“ expr “+” expr “)” | digits)“+” expr “)” | digits
- Continua tendo expr’s do lado direito!

Sintaxe de uma Linguagem de Programação

- As abreviações não acrescentam a ERs o poder de expressar recursão
- É isso que precisamos para expressar a recursão mútua entre sum e expr
- E também para expressar a sintaxe de linguagens de programação

Solução

- <u>Gramáticas livre de contexto</u>
  - Reconhecimento realizado por autômatos finitos equipados com uma pilha
  - Reconhecimento realizado por algoritmos recursivos descendentes

Gramáticas

- O uso de gramáticas traz vantagens para projetistas de linguagens e escritores de compiladores pelas seguintes razões:
  - Uma gramática dá uma especificação sintática precisa de uma LP
  - Para certas classes de gramáticas, pode-se automatizar o processo de construção do analisador sintático e o gerador automático pode revelar certas ambiguidades sintáticas da LP difíceis de serem detectadas pelo escritor do compilador
  - Uma gramática bem projetada dá estrutura à uma LP, o que facilita a compilação e a detecção de erros de programas fonte
  - Novas construções sintáticas que surgem com a evolução de uma LP podem ser incorporadas mais facilmente à linguagem se seu compilador tem uma implementação baseada em uma descrição gramatical

Gramáticas

- Descrevem a sintaxe das construções de uma linguagem de programação, como expressões e comandos

- Exemplo:
  - A estrutura do comando condicional:
    - stmt → **if** *(expr) stmt* **else** *stmt*
    - *stmt* representa comandos
    - *expr* representa expressões

Gramáticas Livre de Contexto

- Formalmente, uma gramática livre de contexto possui quatro componentes:
  1. **Elementos terminais**: símbolos do alfabeto da linguagem. Ex.: if, (,), else
  2. **Elemento não-terminais**: variáveis que representam sequências de terminais (e/ou não-terminais).
  3. Designação de um **símbolo inicial** da gramática;
  4. Um **conjunto de produções**, que especificam a forma como os terminais e os não-terminais podem ser combinados para forma cadeias. Cada produção consiste em:
     - Um não-terminal (cabeça ou lado esquerdo da produção)
     - Um símbolo → ou ::=;
     - Um corpo ou lado direito da produção, que consiste em zero ou mais terminais e não-terminais

Gramáticas Livre de Contexto

- **Exemplo**
  - Numa gramática para expressões contendo operações aritméticas simples:
    1. Os símbolos terminais são: id + - * / ( )
    2. Os símbolos não-terminais são: E, T e F
    3. O símbolo inicial é E
    4. Produções: E → E+T | E–T | T

       T → T*F | T/F | F

       F → (E) | id

Gramáticas Livre de Contexto

- Construa uma gramática para a linguagem a seguir
  - $L = \{ 0^n 1^n \mid n >= 1\}$

- Gramática G
  - Conjunto de símbolos terminais = {0,1}
  - Conjunto de símbolos não terminais = {S}
  - Símbolo inicial = S
  - Conjunto de produções
    - $S \rightarrow 01$
    - $S \rightarrow 0S1$

Gramáticas Livre de Contexto

- Construa uma gramática para a linguagem a seguir
  - $L = \{ ww^R | w \in \{a,b\}^* \}$

- Gramática $G = (\{S\}, \{a,b\}, S, P)$ tal que
  - $P = \{ S \rightarrow aSa; S \rightarrow bSb; S \rightarrow \varepsilon \}$
  - As cadeias $\{\varepsilon$, aa, bb, abba, aabbaa, abbbba ... $\}$ pertencem a L

Gramáticas Livre de Contexto

- Dever ser precisa e fácil de entender
  - Precisa: não ambígua
  - Fácil de entender: deixa evidente a sintaxe da linguagem

- Historicamente é um formalismo adequado
  - Fácil de especificar/manter/entender

Derivação

- O processo de derivação consiste na aplicação de regras de reescrita:
  - Começa com o símbolo inicial e depois vai substituindo repetidamente um não-terminal pelo corpo de uma de suas produções
  - As cadeias de terminais que podem ser derivadas a partir do símbolo inicial formam a linguagem definida pela gramática

Derivação

Exercício. Qual das strings a seguir faz parte da linguagem definida pela gramática abaixo?

1. abcba
2. acca
3. aba
4. abcbcba

$$S \rightarrow aXa$$
$$X \rightarrow \varepsilon \mid bY$$
$$Y \rightarrow \varepsilon \mid cXc$$

Derivação

Exercício. Qual das seguintes derivações é válida de acordo com a gramática ao lado?

1 - S
a*X*a
aa

2 - S
a*X*a
ab*Y*a
ac*X*ca
acca

3 - S
a*X*a
ab*Y*a
abc*X*cda
abccda

4 - S
a*X*a
ab*Y*a
abc*X*ca
abcb*Y*caabc
bdca

$$S \rightarrow a\mathit{X}a$$
$$X \rightarrow \varepsilon \mid b\mathit{Y}$$
$$Y \rightarrow \varepsilon \mid c\mathit{X}c \mid d$$

Derivação

- <u>Exemplo</u>:
  - Considere a gramática a seguir com um único não terminal E, que acrescenta a produção E → -E à gramática:
    - E → E+E | E*E | -E | (E) | id
    - Se E representa uma expressão, então –E também deve representar uma expressão. A substituição de um único E por –E é descrita por: E ⇨ -E (E deriva em -E).
    - A partir de um único E podemos aplicar as produções repetidamente em qualquer ordem para obter uma sequência de substituições (derivação).
      - Por exemplo: E ⇨ -E ⇨ -(E) ⇨ -(id)

Derivação

$1. S \rightarrow S; S$
$2. S \rightarrow \text{id} := E$
$3. S \rightarrow \text{print}\ (L)$
$4. E \rightarrow \text{id}$
$5. E \rightarrow \text{num}$
$6. E \rightarrow E + E$
$7. E \rightarrow (S, E)$
$8. L \rightarrow E$
$9. L \rightarrow L, E$

```
id := num; id := id + (id := num + num, id)
```

Derivação

id := num; id := id + (id := num + num, id)

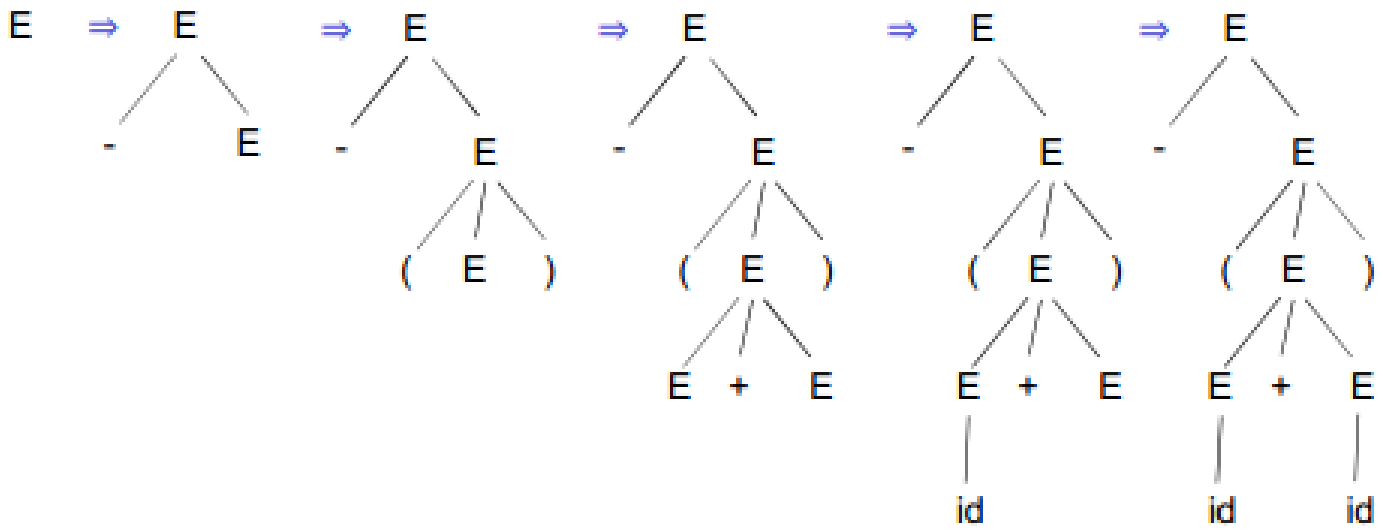
- $\underline{S}$
- S ; $\underline{S}$
- $\underline{S}$; id := E
- id := $\underline{E}$; id := E
- id := num ; id := $\underline{E}$
- id := num ; id := $\underline{E}$ + E
- id := num ; id := $\underline{E}$ + (S, E)
- id := num ; id := id + ($\underline{S}$, E)
- id := num ; id := id + (id := $\underline{E}$, E)
- id := num ; id := id + (id := E + E, $\underline{E}$)
- id := num ; id := id + (id := $\underline{E}$ + E, id)
- id := num ; id := id + (id := num + $\underline{E}$, id)
- id := num ; id := id + (id := num + num, id)

Derivação

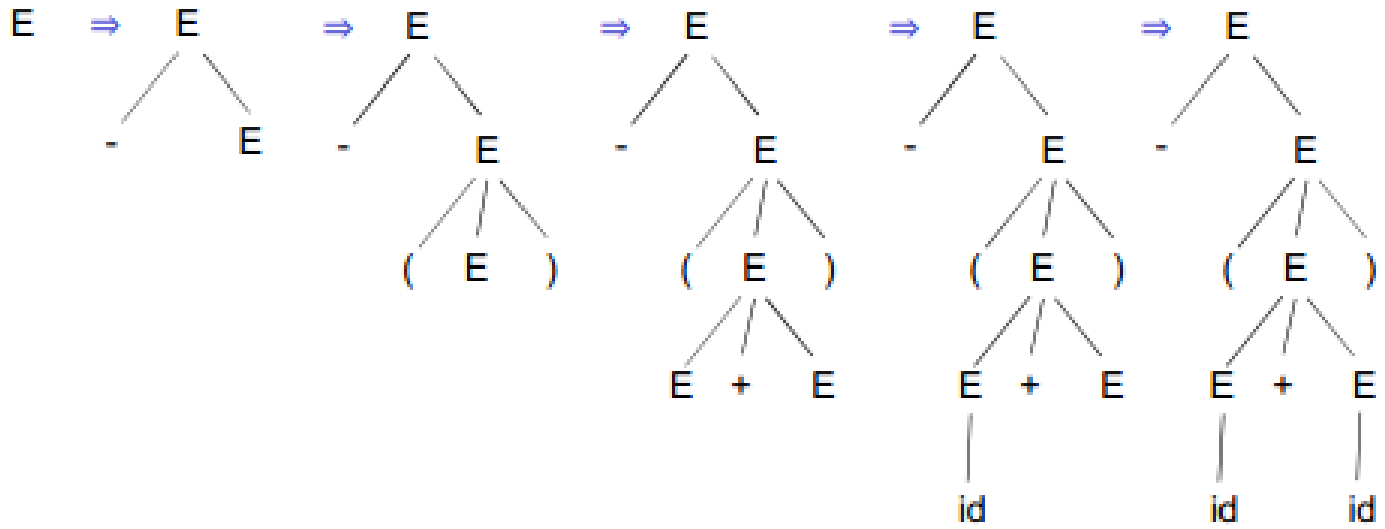
- Podem existir várias derivações para a mesma palavra
- left-most: o não terminal mais a esquerda é sempre o expandido
- right-most: o não terminal mais a direita é sempre o expandido
- Ex: Gramática de parênteses balanceados S → SS|(S)|()
  - derivação esquerda: cadeia (())()
    - S=>SS =>**(S)S =>(())S =>(())()**
  - derivação direita
    - S=>SS =>S**() =>(S)() =>(())()**

Árvores de Derivação

- Representação gráfica de uma derivação que filtra a ordem na qual as produções são aplicadas para substituir não-terminais
- Cada nó interior da uma árvore representa a aplicação de uma produção
- Exemplo:
  - Gramática: E → E+E | E*E | -E | (E) | id
  - Derivação: E ⇨ -E ⇨ -(E) ⇨ -(E+E) ⇨ -(id+E) ⇨ -(id+id)



Árvores de Derivação

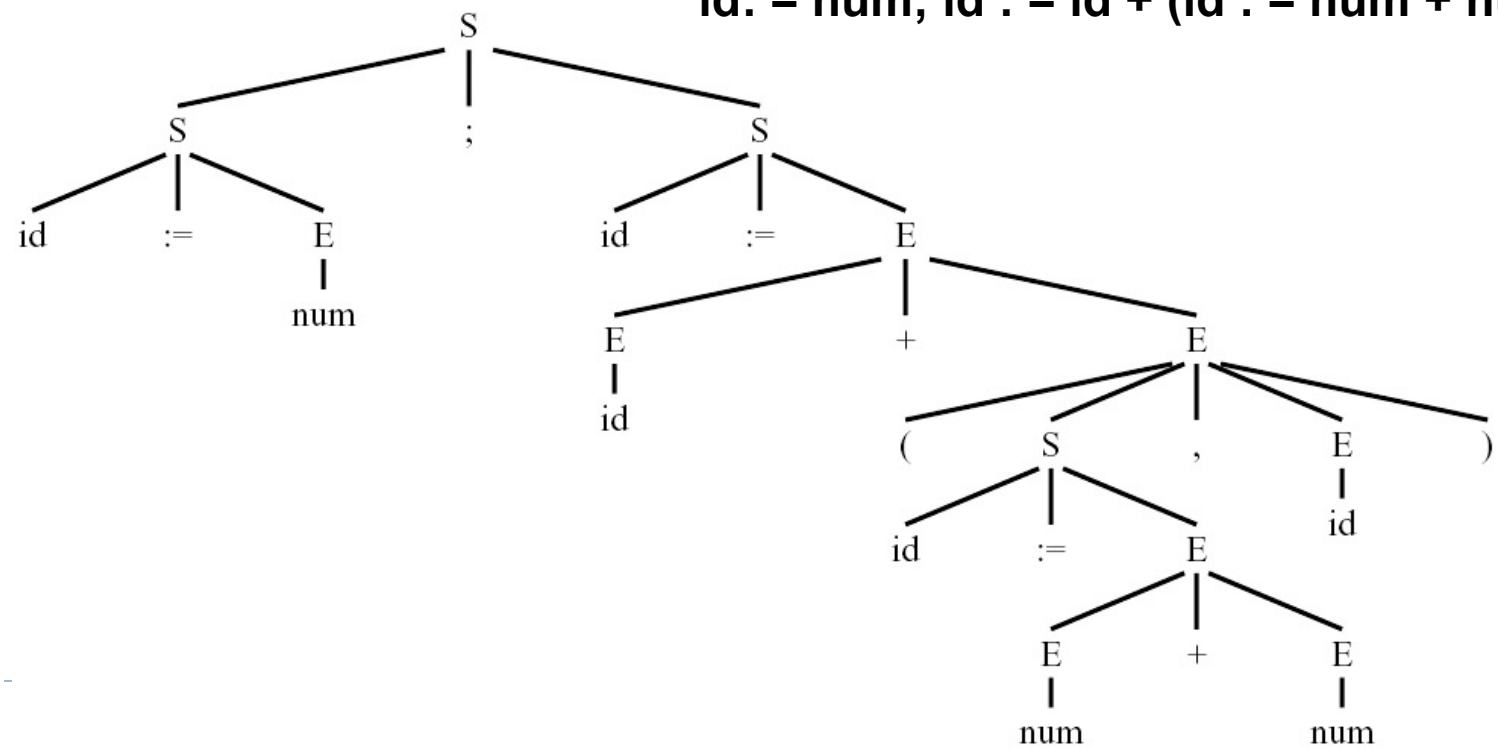


- Da esquerda para a direita, as folhas formam o resultado da árvore, que é a **cadeia gerada** a partir da raiz da árvore
- O processo de encontrar uma árvore de derivação para determinada cadeia de terminais é chamado de **análise** ou **reconhecimento** dessa cadeia

Árvores de Derivação

1. $S \rightarrow S; S$
2. $S \rightarrow \text{id} := E$
3. $S \rightarrow \text{print}\ (L)$
4. $E \rightarrow \text{id}$
5. $E \rightarrow \text{num}$
6. $E \rightarrow E + E$
7. $E \rightarrow (S, E)$
8. $L \rightarrow E$
9. $L \rightarrow L, E$

id: = num; id : = id + (id : = num + num, id)



Exercícios

Dada a gramática:

E -> E + T | E - T | T
T -> T * F | F
F -> (E) | num
num -> 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Escreva derivações e árvores sintáticas para as expressões a seguir:

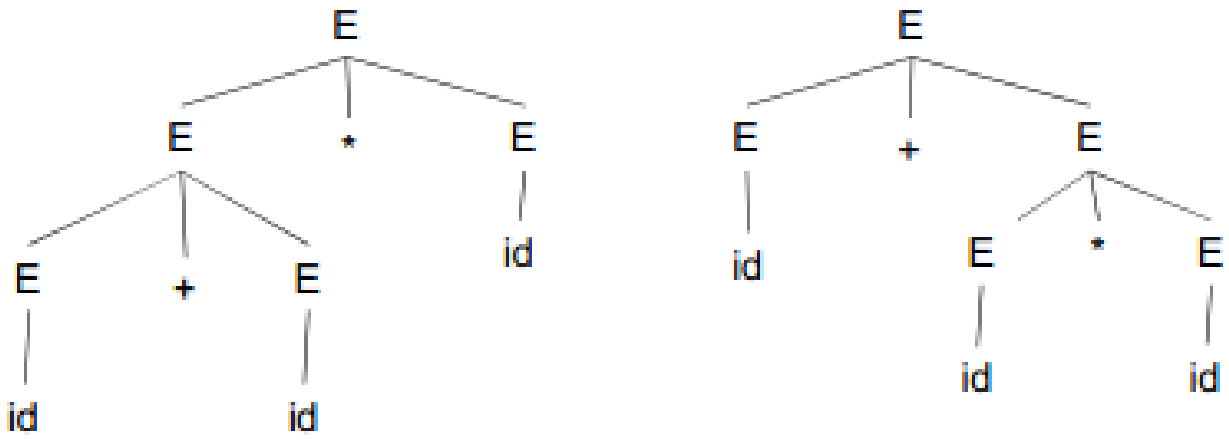
- 3 + 4 * 5 - 6
- 3 * (4 - 5 + 6)
- 3 - (4 + 5 * 6)

Observações

- Veremos que estratégias de análise sintática diferentes levam a derivações mais à esquerda ou mais à direita
- Podemos ter várias derivações para uma mesma cadeia w, mas só pode haver uma árvore sintática
- A árvore sintática dá a estrutura do programa, e a estrutura se traduz no significado do programa
- Um programa com mais de uma árvore sintática tem mais de uma possível interpretação
- A diferença entre uma derivação mais à esquerda e mais à direita se traduz em uma diferença na implementação do analisador sintático, e não na estrutura do programa

Ambiguidade

- Uma gramática que produz mais de uma árvore de derivação para alguma sentença é considerada uma gramática ambígua
- Exemplo:
  - A gramática de expressões aritméticas permite duas derivações para a sentença **id+id*id**
  - E → E+E | E*E | -E | (E) | id



Ambiguidade

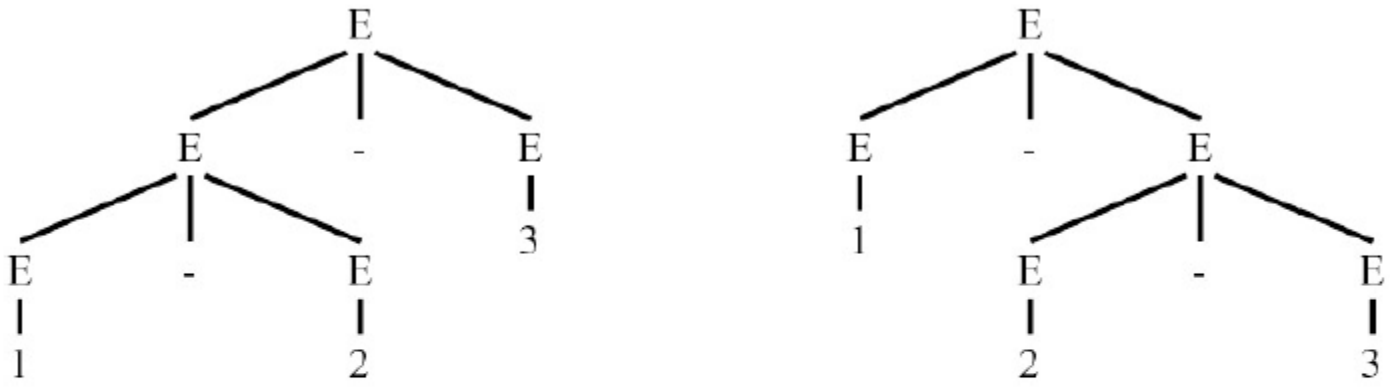
$E \rightarrow \text{id}$
$E \rightarrow \text{num}$
$E \rightarrow E * E$
$E \rightarrow E / E$
$E \rightarrow E + E$
$E \rightarrow E - E$
$E \rightarrow (E)$

Construa Parse Trees para as seguintes expressões:

1-2-3

1+2*3

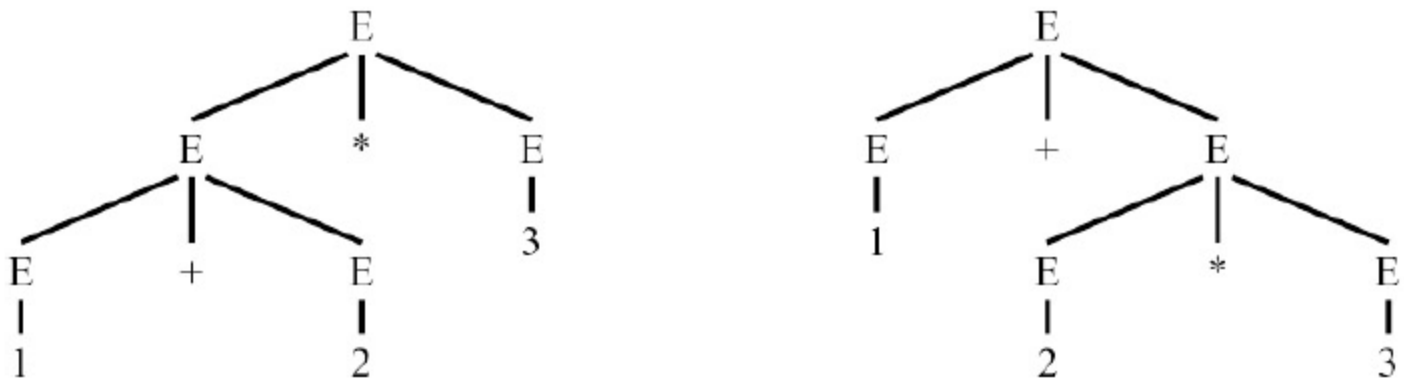
Ambiguidade



Ambígua!

(1-2)-3 = -4 e 1-(2-3) = 2

Ambiguidade



Ambígua!

(1+2)*3 = 9 e 1+(2*3) = 7

Ambiguidade

- Ambiguidade é ruim para uma linguagem de programação, pois leva a interpretações inconsistentes entre diferentes compiladores
- Infelizmente, não existe um algoritmo para detectar se uma gramática qualquer é ambígua ou não
- Mas existem heurísticas, a principal delas é verificar se existe uma regra misturando recursão à esquerda e recursão à direita
- Geralmente podemos transformar uma gramática para retirar a ambiguidade

Ambiguidade

- A gramática de expressões:
  - A ambiguidade vem da gramática não estar levando em conta as regras de associatividade e precedência dos operadores
  - Cada nível de precedência tem que ganhar seu próprio não-terminal
  - Operadores que devem ser associativos à esquerda precisam usar recursão à esquerda, e associativos à direita precisam de recursão à direita
    - (1-2)-3 e não 1-(2-3)

Ambiguidade

$E \rightarrow E + T$ $\quad$ $T \rightarrow T * F$ $\quad$ $F \rightarrow \text{id}$

$E \rightarrow E - T$ $\quad$ $T \rightarrow T / F$ $\quad$ $F \rightarrow \text{num}$

$E \rightarrow T$ $\quad$ $T \rightarrow F$ $\quad$ $F \rightarrow (E)$

Construa as derivações e Parse Trees para as seguintes expressões:

1-2-3

1+2*3

Ambiguidade

Considere o fragmento de gramática abaixo apresentado. Mostre, através do exemplo abaixo, o problema típico de determinadas linguagens de alto nível – ambiguidade na construção de comandos condicionais aninhados. Use as árvores de derivação.

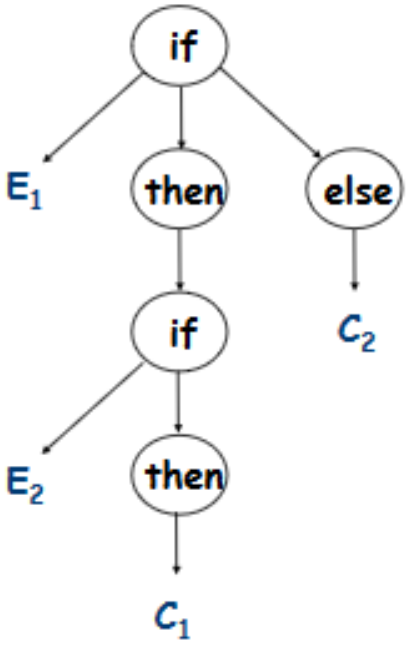
programa → ... <comando> ...
<comando> → <condicional>
<condicional> → if <expressão> then <comando>
<condicional> → if <expressão> then <comando> else <comando>
<expressão> → ....

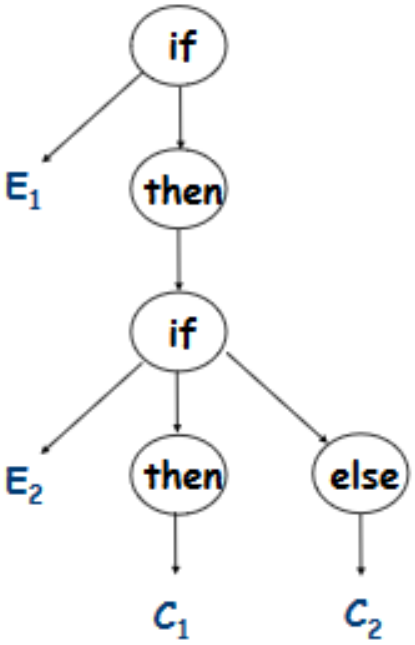
Exemplo: if <expressão> then if <expressão> then <comando> else <comando>

Ambiguidade

Essa forma sentencial tem duas derivações

<u>if</u> $Expr_1$ <u>then</u> <u>if</u> $Expr_2$ <u>then</u> Cmd_1 <u>else</u> Cmd_2





Ambiguidade

- A gramática G

$$S \rightarrow SS \mid (S) \mid \varepsilon$$

É ambígua? Porquê?

Estratégias de Análise Sintática

- Os analisadores sintáticos podem ser classificados:
  - Universal
  - Descendente (Top-Down)
  - Ascendente (Bottom-Up)

Estratégias de Análise Sintática

- Universal
  - Existem métodos universais de análise sintática, como o algoritmo de Cocke-Younger-Kasami e o de Earley
    - Trabalham com qualquer tipo de gramática livre de contexto
    - Mas são ineficientes para se usar na produção de compiladores, pois são de ordem de $O(n^3)$, com n sendo o tamanho da cadeia de tokens
  - Earley, em 1970, apresentou um método de A.S. Top-Down sem Backtracking que reconhece todas as GLCs com $O(n^3)$
    - Se elas forem não ambíguas, o tempo é $O(n^2)$ e chega a ser $O(n)$ para muitas linguagens.
    - Detalhes: ww.cs.uiowa.edu/~RVS/Courses/Compiler/Notes/earley.pdf

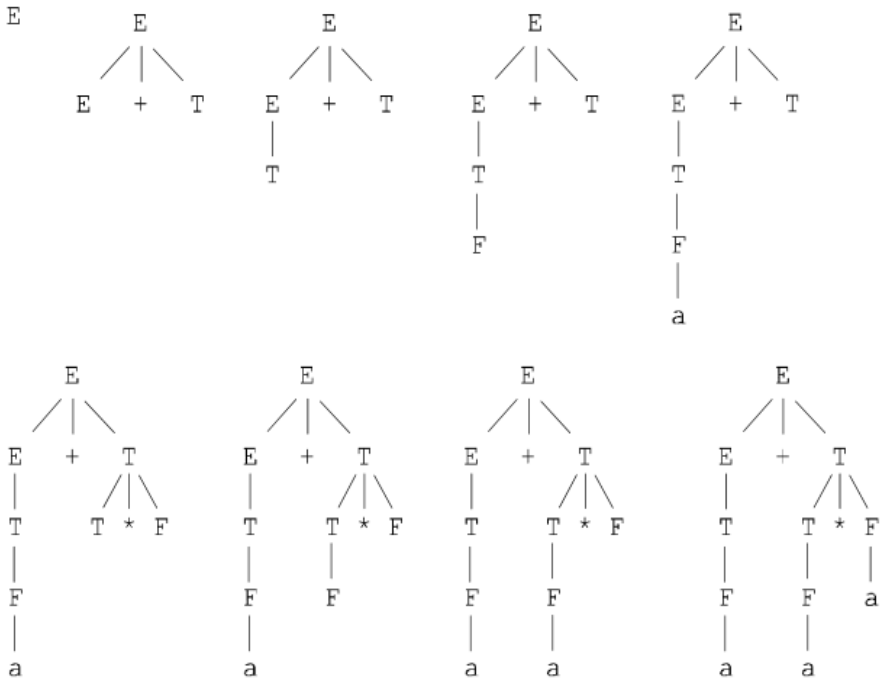
Estratégias de Análise Sintática

- Os métodos Top-Down e Bottom-Up mais eficientes e interessantes são determinísticos (O(n))
  - Eles trabalham somente com subclasses de GLCs
  - Muitas dessas subclasses, por exemplo, as gramáticas LL(k) e LR(k) são bastante expressivas para descrever a maioria das linguagens de programação

Estratégias de Análise Sintática

- Um analisador sintático descendente tenta construir a árvore de derivação para uma sentença w a partir do símbolo inicial, aplicando regras de produção até produzir todos os símbolos de w

```
E → E + T
E → T
T → T * F
T → F
F → ( E )
F → a
```



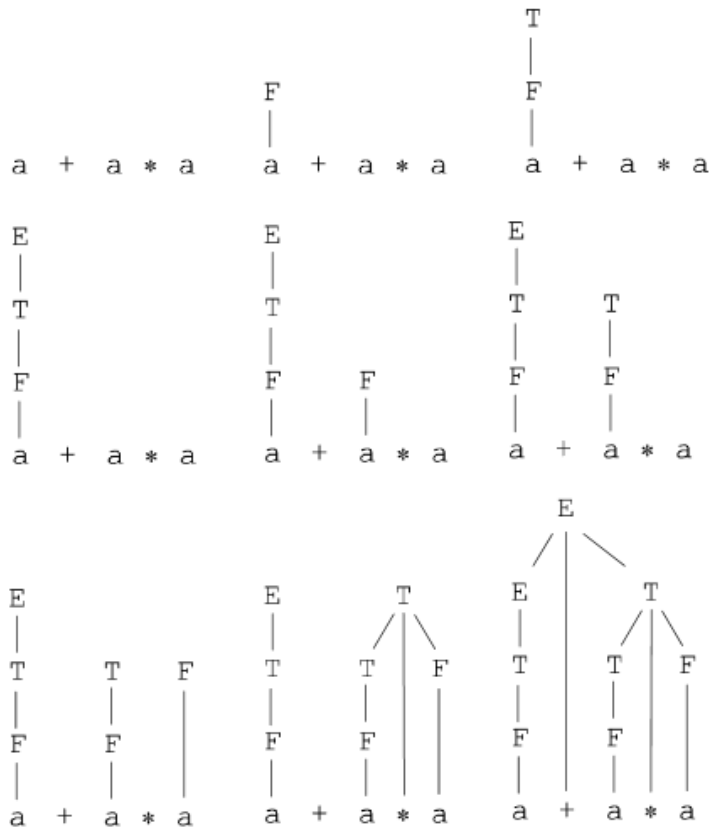
Estratégias de Análise Sintática

- Analisador sintático descendente (ASD):
  - Os nós da árvore são criados em pré-ordem (busca em profundidade)
  - É um método que produz uma derivação mais a esquerda para uma cadeia de entrada
  - Problema: qual produção deve ser aplicada a um não terminal?
  - Quando uma produção é escolhida, o restante do processo de análise consiste em casar os símbolos terminais da produção com o a cadeia de entrada
  - Tipos: ASD com retrocesso e AS preditivo (recursivo e não recursivo)

Estratégias de Análise Sintática

- Um analisador sintático ascendente tenta construir a árvore de derivação para uma sentença w a partir dos símbolos de w, fazendo reduções (substituir o lado direito de uma regra pelo seu lado esquerdo) até obter o símbolo inicial

$$E \rightarrow E + T$$
$$E \rightarrow T$$
$$T \rightarrow T * F$$
$$T \rightarrow F$$
$$F \rightarrow (E)$$
$$F \rightarrow a$$



Analisador Sintático Descendente Com Retrocesso

- É o mais antigo método de construção de ASD
- Usa tentativa e erro, tenta derivar uma sentença esgotando todas as possíveis opções de derivação
- Construir uma árvore sintática a partir da raiz criando nós em pré-ordem
- Características:
  - Exploratório: tenta todas as possibilidades
  - Ineficiente

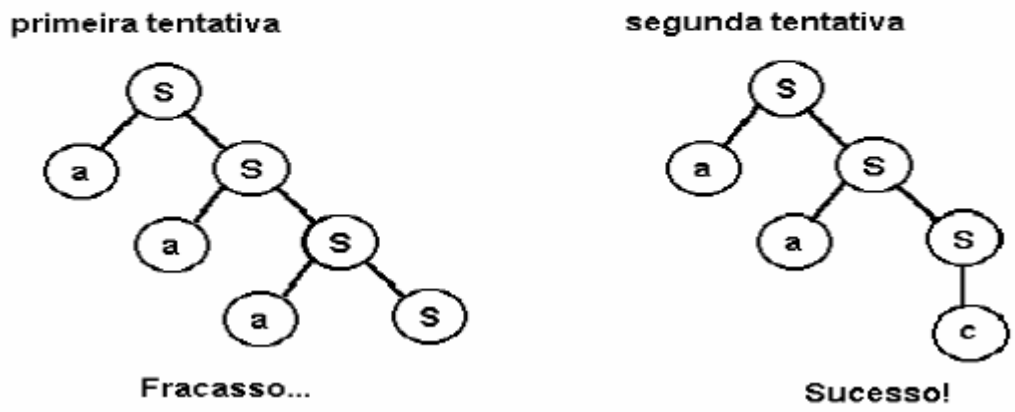
Analisador Sintático Descendente Com Retrocesso

- Funcionamento:
  - A cada passo, escolhe uma regra e aplica
  - Se falhar em algum ponto, retrocede e escolhe uma outra regra
  - O processo termina quando a cadeia é reconhecida ou quando as regras se esgotaram e a cadeia não foi reconhecida

Ex:

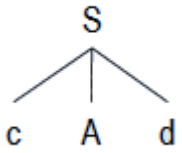
S ::= aS | c

w = aac

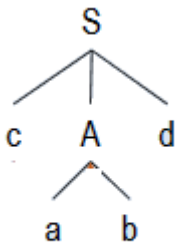


Analisador Sintático Descendente Com Retrocesso

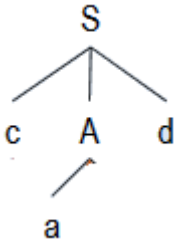
- Exemplo: Considere a gramática abaixo
  - S → cAd
  - A → ab | a
  - Para a cadeia w = cad temos:



Expansão de S



Expansão de A com primeira produção



Expansão de A com segunda produção

Analisador Sintático Descendente Com Retrocesso

- O registro de que regra de produção foi adotada em determinado ponto da análise, que porção da sentença analisada já foi lida, etc. é mantido em uma estrutura tipo pilha para facilitar o mecanismo de tentativa e erro

- O número de derivações pode ser uma função exponencial do tamanho da cadeia

- Por gastar muita memória para registrar os passos que adota durante a análise de uma sentença, além de ser demorado, este método praticamente não é mais usado na construção de compiladores

Analisador Sintático Preditivo Recursivo

- Para quem dispõe de uma linguagem com recursividade para a implementação de um compilador, este método é o mais indicado pois aproveita a estrutura da gramática. Porém a gramática deve:
  - Não apresentar recursão à esquerda

    $S \rightarrow Aa \mid b$

    $A \rightarrow Ac \mid \varepsilon$

  - Suas produções estejam adequadamente fatoradas: para um não terminal qualquer, não devem existir duas regras que comecem com um mesmo terminal

    $S \rightarrow aAb \mid abC \mid b$

    $A \rightarrow cb \mid ca \mid cC$

    $C \rightarrow a \mid b \mid c$

- O analisador preditivo recursivo é escrito na forma de um conjunto de procedimentos, sendo associado a cada procedimento um elemento não terminal da gramática

Analisador Sintático Preditivo Recursivo

- Por exemplo, dada a gramática:

  <expr> ::= <termo> + <expr> | <termo>

  <termo> ::= <fat or> * <termo> | <fator>

  <fator> ::= <primário> ** <fator> | <primário>

  <primário> ::= IDENT | NÚMERO | (<expr>)

- Podemos interpretar a regra (1) como: "expressão é um termo seguido de zero ou mais expressões conectadas pelo operador +", ou seja:

  expressão = termo

  expressão = termo + expressão

  expressão = termo + expressão + expressão

  e assim por diante...

Analisador Sintático Preditivo Recursivo

- Podemos reescrevê-la na forma abaixo, usando uma metalinguagem onde {w}* significa zero ou mais ocorrências de w:

(1) <expr> ::= <termo> { + <expr> }*
(2) <termo> ::= <fator> { * <termo> }*
(3) <fator> ::= <primário> { ** <fator> }*
(4) <primário> ::= IDENT | NÚMERO | (<expr>)

- Podemos, agora, escrever um algoritmo que faça a análise de sentenças geradas a partir dessa gramática

Analisador Sintático Preditivo Recursivo

```
Procedimento analisador sintático {
    obtenha_símbolo(); /* chama o léxico */
    EXPR();
}

Procedimento EXPR() {
    TERMO();
    se símbolo_lido = '+' então {
        obtenha_símbolo();
        EXPR();
    }
}

Procedimento TERMO() {
    FATOR();
    se símbolo_lido = '*' então {
        obtenha_símbolo();
        TERMO();
    }
}
```

<expr> ::= <termo> { + <expr> }*

<termo> ::= <fator> { * <termo> }*

Analisador Sintático Preditivo Recursivo

```
Procedimento FATOR() {
    PRIMÁRIO();
    se símbolo_lido = '**' então {
        obtenha_símbolo();
        FATOR();
    }
}
```

<fator> ::= <primário> { ** <fator> }*

```
Procedimento PRIMÁRIO() {
    se símbolo_lido = IDENT então {
        /* trate adequadamente um identificador */
        obtenha_símbolo();
    }
    senão se símbolo_lido = NÚMERO então {
        /* trate adequadamente um número */
        obtenha_símbolo();
    }
    senão se símbolo_lido = '(' então {
        obtenha_símbolo();
        EXPR();
        se símbolo ≠ ')' então
            ERRO( "falta )" );
        senão
            obtenha_símbolo();
    }
}
```

<primário> ::= IDENT | NUMERO | (<expr>)

Analisador Sintático Preditivo Recursivo

- Exemplo, dada a gramática:

<S> ::= a<A>d

<A> ::= c<A> | e<B>

<B> ::= f | g

Construir o Analisador sintático preditivo recursivo

```
procedimento  ASD_Recursivo()
        obtenha_simbolo()
        S()
        se (terminou_cadeia) então
                SUCESSO
        senão ERRO
```

Analisador Sintático Preditivo Recursivo

```
<S> ::= a<A>d
S()
    se (simbolo = 'a') então
            obtenha_simbolo();
            A()
            se (simbolo = 'd') então
                obtenha_simbolo()
            senão ERRO
    senão ERRO
```

```
<A> ::= c<A> | e<B>
A()
    se (simbolo='c') então
            obtenha_simbolo()
            A()
    senão se (simbolo ='e') então
            obtenha_simbolo()
            B()
    senão ERRO
```

```
<B> ::= f | g
B()
        se (simbolo='f') ou (simbolo='g') então
            obtenha_simbolo()
        senão ERRO
```

Problema da Recursividade

- Recapitulando: para que se possa construir um analisador sintático preditivo, uma gramática não pode apresentar:
  - Recursão à esquerda

    $S \rightarrow Aa \mid b$

    $A \rightarrow Ac \mid \varepsilon$

  - Produções não adequadamente fatoradas a esquerda

    $S \rightarrow aAb \mid abC \mid b$

    $A \rightarrow cb \mid ca \mid cC$

    $C \rightarrow a \mid b \mid c$

Problema da Recursividade

- Uma gramática é recursiva à esquerda se $\exists A \in NT$ tal que $\exists$ uma derivação $A \Rightarrow A\alpha$, para alguma string $\alpha \in (NT \cup T)^+$

- Nesse caso, qualquer algoritmo que implemente um analisador preditivo recursivo vai entrar em ciclo ("loop") infinito

Problema da Recursividade

- Podemos resolver o problema da recursividade direta à esquerda com uma transformação muito simples na gramática. Regras do tipo:

$$A ::= A\alpha \mid \beta$$

- Cujo objetivo é produzir cadeias da forma:

$$\beta, \beta\alpha, \beta\alpha\alpha, \ldots$$

- Devem ser transformadas em:

$$A ::= \beta A'$$
$$A' ::= \alpha A' \mid \varepsilon$$

Problema da Recursividade

- Transformar produções do tipo $A ::= A\alpha \mid \beta$ em:

$$A ::= \beta A'$$
$$A' ::= \alpha A' \mid \varepsilon$$

- Generalizando:

$$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \ldots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \ldots \mid \beta_m$$

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \ldots \mid \beta_m A'$$
$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \ldots \mid \alpha_n A' \mid \varepsilon$$

Problema da Recursividade

- Exemplo: Gramática para expressões simples:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid id$

$A \rightarrow A\alpha \mid \beta$

❑A substituição abaixo elimina a recursão imediata à esquerda:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \varepsilon$

Eliminando a recursão à esquerda para E e T:

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \varepsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \varepsilon$

$F \rightarrow (E) \mid id$

Problema da Recursividade

- Exercício: Eliminar a recursividade à esquerda da gramática:

$S \rightarrow (L) \mid a$

$L \rightarrow L,S \mid S$

$A \rightarrow A\alpha \mid \beta$

❑A substituição abaixo elimina a recursão imediata à esquerda:

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \varepsilon$

$S \rightarrow (L) \mid a$

$L \rightarrow S\ L'$

$L' \rightarrow ,\ S\ L' \mid \varepsilon$

Problema da Recursividade

- Outra restrição diz respeito à ocorrências de produções do tipo:

$$A ::= \alpha\beta$$

$$A ::= \alpha\gamma$$

- Qual regra aplicar a partir do ponto A na árvore de derivação sintática de uma dada sentença?
  - Podemos aplicar mais de uma regra para chegarmos ao mesmo resultado

- Como resolver esse problema?
  - Aplicando uma transformação nas regras de produção da gramática: fatoração à esquerda

$$A ::= \alpha C$$

$$C ::= \beta \mid \gamma$$

Problema da Recursividade

- Exemplo: Considere a gramática G a seguir:

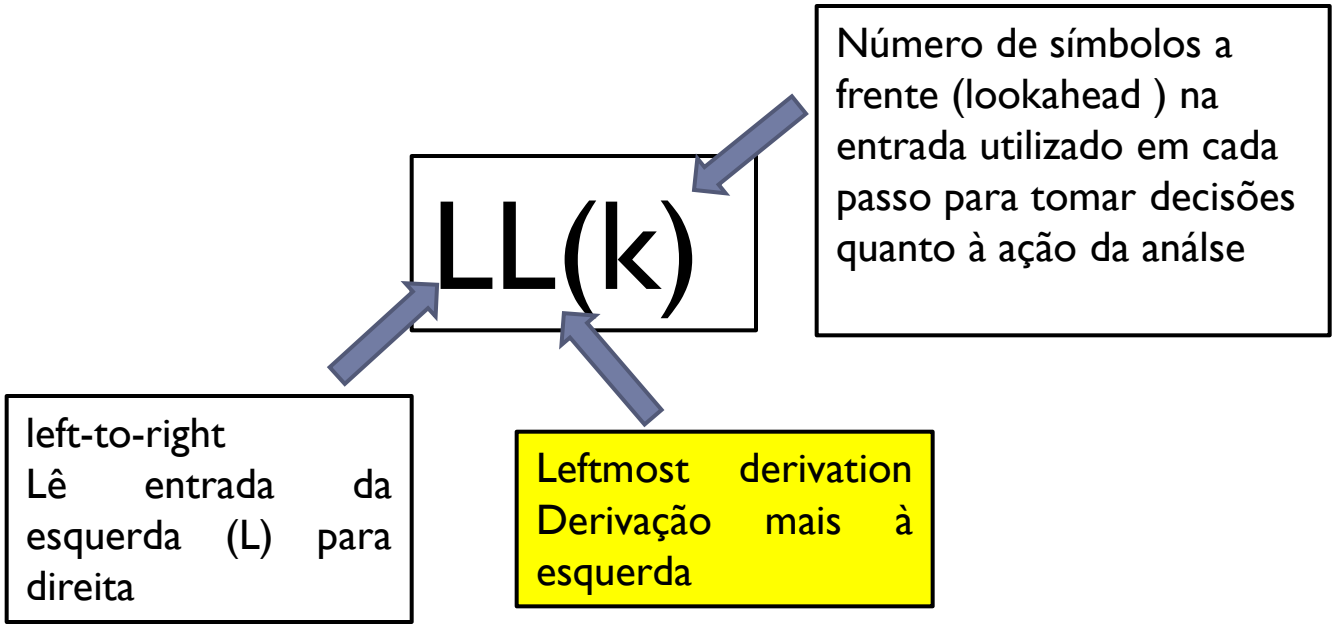
$$S \rightarrow aAb \mid abC \mid b$$
$$A \rightarrow cb \mid ca \mid cC$$
$$C \rightarrow a \mid b \mid c$$

Obtenha uma gramática equivalente à G, fatorada à esquerda

$$S \rightarrow aS' \mid b$$
$$S' \rightarrow Ab \mid bC$$
$$A \rightarrow cC$$
$$C \rightarrow a \mid b \mid c$$

Analisador Sintático Preditivo Sem Recursão

- Também chamado de Analisador de Gramáticas LL(K)



Analisador Sintático Preditivo Sem Recursão

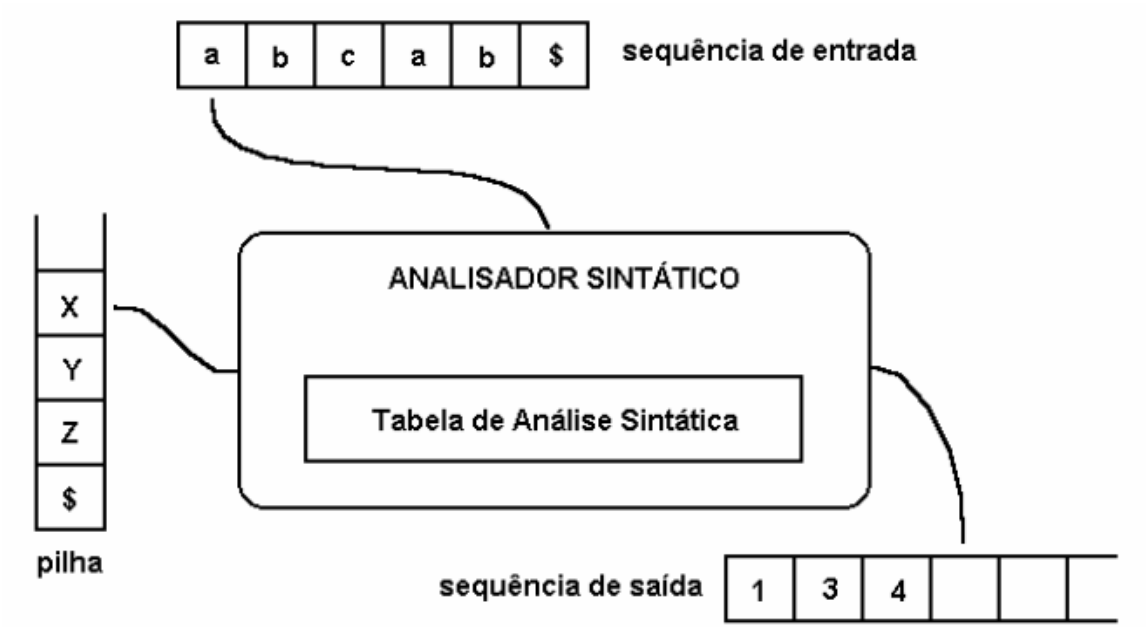
- A ideia do analisador LL(K) é de que basta olharmos no máximo K símbolos à frente na sentença, a partir do ponto em que estamos na ADS, para que possamos decidir que regra de produção aplicar

```
Exemplo:

    S ::= aS | bS | c          S ::= abS | acS | ad

    G(S) é LL(1)               G(S) é LL(2)

    w = abc                    w = abad
    w = bac                    w = acad
```

- Em termos de linguagens de programação, quase sempre é possível obter-se uma gramática LL(1) que permita o reconhecimento sintático de programas através de um analisador LL(1) que é bastante simples de implementar

Analisador Sintático Preditivo Sem Recursão

- o analisador sintático Preditivo Sem Recursão:
  - Receberá uma sequência de entrada (a sentença a ser analisada)
  - Manipulará uma estrutura de dados tipo pilha (onde monta a ADS)
  - Consultará uma tabela de análise sintática (tabela de "parsing")
  - Emitirá uma sequência de saída (regras que estão sendo aplicadas)



Analisador Sintático Preditivo Sem Recursão

- A sequência de entrada é formada pela sentença a ser analisada, seguida por um símbolo delimitador ($)
- A pilha contém uma sequência de símbolos da gramática, precedida pelo indicador de base de pilha ($)
  - Inicialmente a pilha contém o símbolo inicial da gramática precedido por $
- A tabela de análise sintática é uma matriz M[A,a] onde 'A' é um não-terminal e 'a' é um terminal ou $
- A sequência de saída constará das produções aplicadas a partir do símbolo inicial, na geração da sentença

Analisador Sintático Preditivo Sem Recursão

- O analisador sintático, a partir de X, símbolo do topo da pilha, e próximo_símbolo, o atual símbolo da entrada, determina sua ação que pode ser uma das quatro possibilidades a seguir:
  1. Se X é um terminal = próximo_símbolo = $, o analisador encerra sua atividade e comunica fim da análise sintática com sucesso
  2. Se X é um terminal = próximo_símbolo ≠ $, o analisador elimina X do topo da pilha e avança para o próximo símbolo de entrada
  3. Se X é um terminal ≠ próximo_símbolo, o analisador acusa um erro de sintaxe (chama rotina de tratamento de erros)
  4. Se X é um não-terminal, o analisador consulta M[X, próximo_símbolo]. Se a resposta for uma regra de produção X ::= MVU, o analisador desempilha X do topo da pilha e empilha UVM (com M no topo da pilha). Para a saída é enviada a regra de produção usada. Se M[X, próximo_símbolo] = ERRO, o analisador acusa um erro de sintaxe (chama rotina de tratamento de erro)

Analisador Sintático Preditivo Sem Recursão

- Exemplo: Seja a gramática G abaixo com a respectiva tabela de análise sintática.

(1) S ::= aAS

(2) S ::= b

(3) A ::= a

(4) A ::= bSA

	a	b	$
S	1	2	ERRO
A	3	4	ERRO

Analisador Sintático Preditivo Sem Recursão

(1) S ::= aAS

(2) S ::= b

(3) A ::= a

(4) A ::= bSA

	a	b	$
S	1	2	ERRO
A	3	4	ERRO

Dada a sentença w = abbab$, o analisador sintático assumiria as seguintes configurações durante a análise:

ENTRADA	PILHA	SAÍDA
abbab$	$S	
abbab$	$SAa	1
bbab$	$SA	1
bbab$	$SASb	1 4
bab$	$SAS	1 4
bab$	$SAb	1 4 2
ab$	$SA	1 4 2
ab$	$Sa	1 4 2 3
b$	$S	1 4 2 3
b$	$b	1 4 2 3 2
$	$	1 4 2 3 2

Analisador Sintático Preditivo Sem Recursão

(1) S ::= aAS

(2) S ::= b

(3) A ::= a

(4) A ::= bSA

	a	b	$
S	1	2	ERRO
A	3	4	ERRO

Exercício: mostre as configurações que o analisador sintático preditivo sem recursão assumiria durante a análise da sentença w = abb$.

Entrada	Pilha	Saída
abb$	$S	
abb$	$SAa	1
bb$	$SA	1
bb$	$SASb	1 4
b$	$SAS	1 4
b$	$SAb	1 4
$	$SA	1 4 2
$	$SA	1 4 2 Erro

Analisador Sintático Preditivo Sem Recursão

S → cAa

A →cB | B

B → bcB | ε

	a	b	c	$
S	ERRO	ERRO	**S → cAa**	ERRO
A	**A → B**	**A → B**	**A → cB**	ERRO
B	**B → ε**	**B →bcB**	ERRO	ERRO

Exercício: mostre as configurações que o analisador sintático preditivo sem recursão assumiria durante a análise das sentenças w1 = cbca$ e w2 = ccbcaa$.

Analisador Sintático Preditivo Sem Recursão

- Vejamos o algoritmo do analisador sintático preditivo sem recursão

```
início
    /* seja X o símbolo do topo da pilha e
        próximo_símbolo o símbolo atual da entrada */

    enquanto X ≠ $ faça {
        se X é terminal então
            se X = próximo_símbolo então {
                elimine X do topo da pilha;
                leia_próximo_símbolo();
            }
            senão ERRO();
        senão se M[X, próximo_símbolo] = "X ::= Y1Y2...Yk então {
            elimine X do topo da pilha;
            empilhe Yk, ..., Y2, Y1
        }
        senão ERRO();
    }

    se próximo_símbolo ≠ $ então ERRO();
fim
```

FIRST E FOLLOW

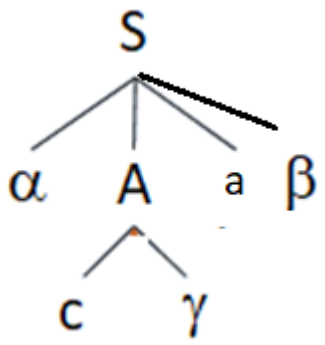
- A ideia e o algoritmo do analisador preditivo sem recursão é bastante simples. Olhando com mais calma, porém, vemos que está faltando uma coisa fundamental: como obter a tabela de análise sintática?

- Para chegarmos até ela, precisamos introduzir dois novos conceitos (ou relações) em gramáticas. São os conceitos de Primeiro (“FIRST”) e Seguidor ("FOLLOW")
  - Estes conceitos auxiliam na escolha de qual produção aplicar, com base no próximo símbolo de entrada

FIRST

- FIRST(α), onde α é qualquer cadeia de símbolos da gramática, é o conjunto de símbolos terminais que iniciam as cadeias derivadas de α
  - Se α =>* ε, então vazio também está em FIRST(α)

- Exemplo: A =>* cγ, portanto c está em FIRST(A)



FIRST

- Para calcular FIRST(X) de todos os símbolos X da gramática, basta aplicar as seguintes regras até que não haja mais terminais ou ε que possam ser acrescentados a algum conjunto FIRST:
  - Se X é um símbolo terminal, então FIRST(X) = {X}
  - Se $X \rightarrow \varepsilon$ é uma produção, então acrescente ε a FIRST(X)
  - Se X é um símbolo não terminal e $X \rightarrow Y_1 Y_2 \ldots Yk$ é uma produção para algum $K \geq 1$, então acrescente α a FIRST(X) se, para algum i, α estiver em FIRST(Y_i), e ε estiver em todos os FIRST(Y_1),..., FIRST(Y_{i-1}); ou seja, $Y_1 \ldots Y_{i-1} \Rightarrow^* \varepsilon$. Se ε está em FIRST(Y_j) para todo j = 1,2,...,k,então adicione vazio a FIRST(X)

FIRST

- Exemplo 1:

COMANDO → CONDICIONAL | ITERATIVO | ATRIBUICAO
CONDICIONAL → if EXP then COMANDO
ITERATIVO → repeat LISTA until EXP | while EXP do COMANDO
ATRIBUICAO → id := EXP

- FIRST(if) = {if}
- FIRST(CONDICIONAL) = {if}
- FIRST (ITERATIVO) = {while,repeat}
- FIRST (ATRIBUICAO) = {id}
- FIRST (COMANDO) = {if, while, repeat, id}

FIRST

- Exemplo 2:

E ::= T E'

E' ::= +TE' | ε

T ::= FT'

T' ::= *FT' | ε

F ::= (E) | id

- FIRST(E) = { (, id }
- FIRST(E') = { +, ε }
- FIRST(T) = { (, id }
- FIRST(T') = { *, ε }
- FIRST(F) = { (, id }

FIRST

- Exemplo 3:

S ::= AB

A ::= c | ε

B ::= cbB | ca

- FIRST(A) = {c, ε }
- FIRST(B) = {c}
- FIRST(S) = {c}

FIRST

- Exemplo 4: alterando a gramática anterior

S ::= AB

A ::= c | ε

B ::= cbB | ca | ε

- FIRST(A) = {c, ε }
- FIRST(B) = {c, ε}
- FIRST(S) = {c, ε}

FIRST

- Exercício 1:

S ::= XYZ

X::= aXb| ε

Y ::= cYcX| d

Z ::= eZYe| f

- FIRST(X) = {a, ε}
- FIRST(Y) = {c,d}
- FIRST(Z) = {e,f}
- FIRST(S) = {a,c,d}

FIRST

- Exercício 2: alterando a gramática anterior

S ::= XYZ

X::= aXb| ε

Y ::= cYcX| d | ε

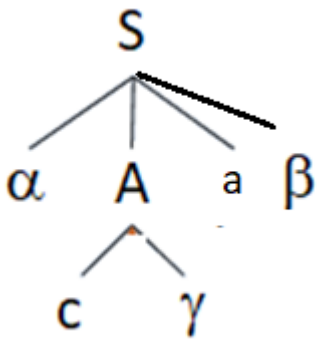
Z ::= eZYe| f | ε

- FIRST(X) = {a, ε}
- FIRST(Y) = {c,d, ε}
- FIRST(Z) = {e,f, ε}
- FIRST(S) = {a,c,d,e,f, ε}

FOLLOW

- FOLLOW é definida para símbolos não-terminais.

- Exemplo: o terminal a está em FOLLOW(A)



FOLLOW

- Sendo A um não terminal, FOLLOW(A) é o conjunto de terminais a que podem aparecer imediatamente à direita de A em alguma forma sentencial. Isto é, o conjunto de terminais a, tal que existe uma derivação da forma S =>*αAaβ para α e β quaisquer.

- Cálculo de FOLLOW:
  - Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)
  - Se existe produção do tipo A $\rightarrow$ αXβ, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)
  - Se existe produção do tipo A $\rightarrow$ αX ou A $\rightarrow$ αXβ, sendo que β =>* ε, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow \vee\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow \vee T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que β =>* ε, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow \vee T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E')
  - FOLLOW(E') contém FOLLOW(E)

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que $\beta \Rightarrow^* \varepsilon$, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow \vee T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que β =>* ε, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow \vee T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T)
  - First(E') = {v, ε}

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que β =>* ε, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow \vee T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T)
  - First(E') = {v, ε}
  - E' pode ser vazio?

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que $\beta \Rightarrow^* \varepsilon$, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow v\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T)
  - First(E') = {v, ε}
  - E' pode ser vazio?
    - FOLLOW(T) contém FOLLOW(E')

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que $\beta \Rightarrow^* \varepsilon$, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow v\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T) = {v,$}

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que β =>* ε, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow v\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T) = {v,$}
- FOLLOW(T')
  - FOLLOW(T') contém FOLLOW(T)

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que $\beta \Rightarrow^* \varepsilon$, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow v\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T) = {v,$}
- FOLLOW(T') = {v,$}

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que β =>* ε, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow v\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T) = {v,$}
- FOLLOW(T') = {v,$}
- FOLLOW(F)

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que $\beta \Rightarrow^* \varepsilon$, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow v\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T) = {v,$}
- FOLLOW(T') = {v,$}
- FOLLOW(F)
  - First(T') = {&, ε}

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que $\beta \Rightarrow^* \varepsilon$, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow v\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T) = {v,$}
- FOLLOW(T') = {v,$}
- FOLLOW(F)
  - First(T') = {&, ε}
  - T' pode ser vazio?

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que $\beta \Rightarrow^* \varepsilon$, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow v\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T) = {v,$}
- FOLLOW(T') = {v,$}
- FOLLOW(F)
  - First(T') = {&, ε}
  - T' pode ser vazio?
    - FOLLOW(F) contém FOLLOW(T')

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que $\beta \Rightarrow^* \varepsilon$, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow v\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

- FOLLOW(E) = {$}
- FOLLOW(E') = {$}
- FOLLOW(T) = {v,$}
- FOLLOW(T') = {v,$}
- FOLLOW(F) = {&, v, $}

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que β =>* ε, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

FOLLOW

- Exercício :

E ::= T E'

E' ::= +TE' | ε

T ::= FT'

T' ::= *FT' | ε

F ::= (E) | id

- FOLLOW(E) = {$,) }
- FOLLOW(E') = {$,)}
- FOLLOW(T) = { +,),$}
- FOLLOW(T') = {+,),$}
- FOLLOW(F) = {*,+,),$}

•Se S é o símbolo inicial da gramática e $ é o marcador de fim da sentença, então $ está em FOLLOW(S)

•Se existe produção do tipo $A \rightarrow \alpha X \beta$, então todos os símbolos de FIRST(β), exceto ε, fazem parte de FOLLOW(X)

•Se existe produção do tipo $A \rightarrow \alpha X$ ou $A \rightarrow \alpha X \beta$, sendo que $\beta \Rightarrow^* \varepsilon$, então todos os símbolos que estiverem em FOLLOW(A) fazem parte de FOLLOW(X)

Gramáticas LL(1)

- Condições necessárias:
  - Sem ambiguidade
  - Sem recursão a esquerda

- Uma gramática G é LL(1) se e somente se, para quaisquer duas produções de G $A \rightarrow \alpha | \beta$
  1. Para um terminal a, tanto α quanto β não derivam cadeias começando com a
  2. No máximo, um dos dois, α ou β, pode derivar a cadeia vazia
     - Das duas regras temos:
       - $First(\alpha) \cap First(\beta) = \emptyset$
       - $\alpha \Rightarrow^* \varepsilon$ implica que $!(\beta \Rightarrow^* \varepsilon)$, e vice-versa
  3. Se $\beta \Rightarrow^* \varepsilon$, então α não deriva nenhuma cadeia começando com um terminal de FOLLOW(A). O mesmo se aplica a $\alpha \Rightarrow^* \varepsilon$, então β não deriva nenhuma cadeia começando com um terminal de FOLLOW(A)
     - $\alpha \Rightarrow^* \varepsilon$ implica que $FIRST(\beta) \cap FOLLOW(A) = \emptyset$.
     - $\beta \Rightarrow^* \varepsilon$ implica que $FIRST(\alpha) \cap FOLLOW(A) = \emptyset$.

Gramáticas LL(1)

- Gramáticas LL(1) podem ser analisadas por analisadores preditivos
  - É possível selecionar a produção apropriada para a ser aplicada em um não terminal examinando-se apenas o símbolo corrente da entrada restante

- Exemplo: a gramática a seguir é LL(1)?

G	→	Expr
Expr	→	Termo + Expr
	|	Termo - Expr
	|	Termo
Termo	→	Fator * Termo
	|	Fator / Termo
	|	Fator
Fator	→	<u>num</u>
	|	<u>id</u>

Gramáticas LL(1)

- É possível transformá-la em um gramática LL(1)?

- Como?

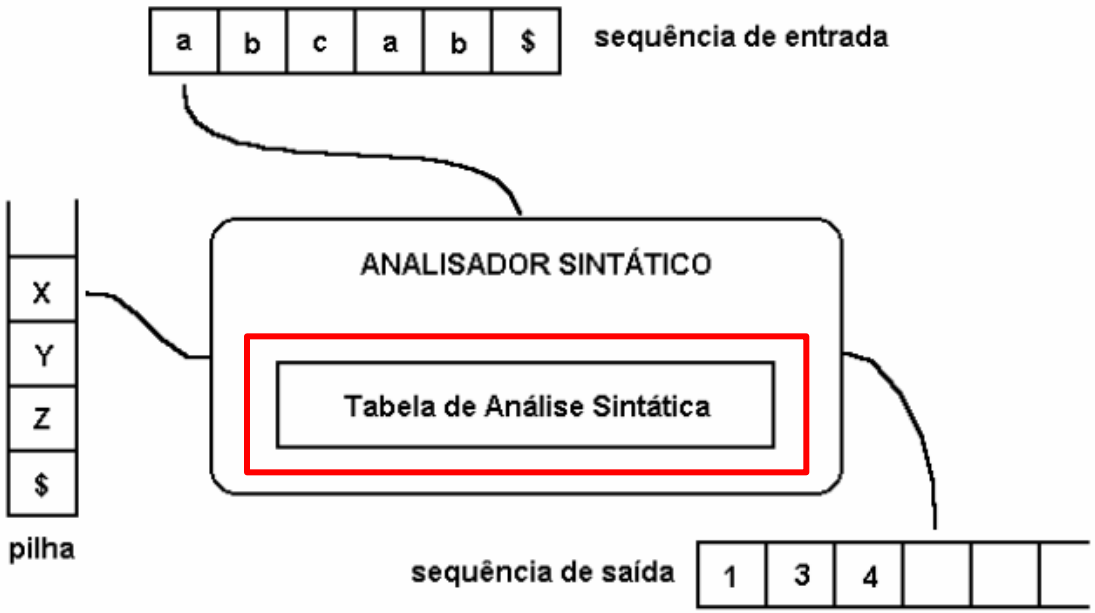
G	→	Expr
Expr	→	Termo + Expr
	|	Termo - Expr
	|	Termo
Termo	→	Fator * Termo
	|	Fator / Termo
	|	Fator
Fator	→	<u>num</u>
	|	<u>id</u>

G	→	Expr
Expr	→	Termo Expr'
Expr'	→	+ Expr
	|	- Expr
	|	ε
Termo	→	Fator Termo'
Termo'	→	* Termo
	|	/ Termo
	|	ε
Fator	→	<u>num</u>
	|	<u>id</u>

Retomando - Analisador Sintático Preditivo Sem Recursão

- O que vimos até aqui?
  - O algoritmo do analisador preditivo sem recursão
  - Conjuntos FIRST e FOLLOW e gramáticas LL(1)

- O que falta para finalizar o estudo deste analisador sintático?



Analisador Sintático Preditivo Sem Recursão

- Como construir a tabela?
  - Reescrever a gramática para satisfazer condições de LL(1) – isso é um pré-requisito
  - Calcular os conjuntos FIRST e FOLLOW
  - Aplicar o seguinte algoritmo:

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

Analisador Sintático Preditivo Sem Recursão

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

$S \rightarrow cAa$	$First(A) = \{b, c, \varepsilon\}$	$Follow(S) = \{\$\}$
$A \rightarrow cb \mid B$	$First(B) = \{b, \varepsilon\}$	$Follow(A) = \{a\}$
$B \rightarrow bcB \mid \varepsilon$	$First(S) = \{c\}$	$Follow(B) = \{a\}$

	a	b	c	$
S				
A				
B				

Analisador Sintático Preditivo Sem Recursão

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

$S \rightarrow cAa$	$First(A) = \{b, c, \varepsilon\}$	$Follow(S) = \{\$\}$
$A \rightarrow cb \mid B$	$First(B) = \{b, \varepsilon\}$	$Follow(A) = \{a\}$
$B \rightarrow bcB \mid \varepsilon$	$First(S) = \{c\}$	$Follow(B) = \{a\}$

	a	b	c	$
S			$S \rightarrow cAa$	
A				
B				

Analisador Sintático Preditivo Sem Recursão

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

$S \rightarrow cAa$	$First(A) = \{b, c, \varepsilon\}$	$Follow(S) = \{\$\}$
$A \rightarrow cb \mid B$	$First(B) = \{b, \varepsilon\}$	$Follow(A) = \{a\}$
$B \rightarrow bcB \mid \varepsilon$	$First(S) = \{c\}$	$Follow(B) = \{a\}$

	a	b	c	$
S			$S \rightarrow cAa$	
A			$A \rightarrow cb$	
B				

Analisador Sintático Preditivo Sem Recursão

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

$S \rightarrow cAa$	$First(A) = \{b, c, \varepsilon\}$	$Follow(S) = \{\$\}$
$A \rightarrow cb \mid B$	$First(B) = \{b, \varepsilon\}$	$Follow(A) = \{a\}$
$B \rightarrow bcB \mid \varepsilon$	$First(S) = \{c\}$	$Follow(B) = \{a\}$

	a	b	c	$
S			$S \rightarrow cAa$	
A		$A \rightarrow B$	$A \rightarrow cb$	
B				

Analisador Sintático Preditivo Sem Recursão

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

$S \rightarrow cAa$	$First(A) = \{b, c, \varepsilon\}$	$Follow(S) = \{\$\}$
$A \rightarrow cb \mid B$	$First(B) = \{b, \varepsilon\}$	$Follow(A) = \{a\}$
$B \rightarrow bcB \mid \varepsilon$	$First(S) = \{c\}$	$Follow(B) = \{a\}$

	a	b	c	$
S			$S \rightarrow cAa$	
A	$A \rightarrow B$	$A \rightarrow B$	$A \rightarrow cb$	
B				

Analisador Sintático Preditivo Sem Recursão

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

$S \rightarrow cAa$	$First(A) = \{b, c, \varepsilon\}$	$Follow(S) = \{\$\}$
$A \rightarrow cb \mid B$	$First(B) = \{b, \varepsilon\}$	$Follow(A) = \{a\}$
$B \rightarrow bcB \mid \varepsilon$	$First(S) = \{c\}$	$Follow(B) = \{a\}$

	a	b	c	$
S			$S \rightarrow cAa$	
A	$A \rightarrow B$	$A \rightarrow B$	$A \rightarrow cb$	
B		$B \rightarrow bcB$		

Analisador Sintático Preditivo Sem Recursão

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

$S \rightarrow cAa$	$First(A) = \{b, c, \varepsilon\}$	$Follow(S) = \{\$\}$
$A \rightarrow cb \mid B$	$First(B) = \{b, \varepsilon\}$	$Follow(A) = \{a\}$
$B \rightarrow bcB \mid \varepsilon$	$First(S) = \{c\}$	$Follow(B) = \{a\}$

	a	b	c	$
S			$S \rightarrow cAa$	
A	$A \rightarrow B$	$A \rightarrow B$	$A \rightarrow cb$	
B	$B \rightarrow \varepsilon$	$B \rightarrow bcB$		

Analisador Sintático Preditivo Sem Recursão

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

$S \rightarrow cAa$	$First(A) = \{b, c, \varepsilon\}$	$Follow(S) = \{\$\}$
$A \rightarrow cb \mid B$	$First(B) = \{b, \varepsilon\}$	$Follow(A) = \{a\}$
$B \rightarrow bcB \mid \varepsilon$	$First(S) = \{c\}$	$Follow(B) = \{a\}$

	a	b	c	$
S	ERRO	ERRO	$S \rightarrow cAa$	ERRO
A	$A \rightarrow B$	$A \rightarrow B$	$A \rightarrow cb$	ERRO
B	$B \rightarrow \varepsilon$	$B \rightarrow bcB$	ERRO	ERRO

Analisador Sintático Preditivo Sem Recursão

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow \vee\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

Analisador Sintático Preditivo Sem Recursão

Para cada produção $A \rightarrow \alpha$ na gramática:

1. Para cada terminal $a \in First(\alpha)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,a]$
2. Se $\varepsilon \in First(\alpha)$

   incluir a produção $A \rightarrow \alpha$ em $M[A,b]$ para cada b terminal em Follow(A)

   Se $\$ \in Follow(A)$ – incluir a produção $A \rightarrow \alpha$ em $M[A,\$]$

Todas as entradas não definidas são erros

- Exemplo :

$E \rightarrow T\ E'$

$E' \rightarrow \vee\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

Para $E \rightarrow T\ E'$ tem-se $FIRST(T\ E') = \{\neg, id\}$

Para $E' \rightarrow \vee\ T\ E'$ tem-se $FIRST(\vee\ T\ E') = \{\vee\}$

Para $E' \rightarrow \varepsilon$ tem-se $FOLLOW(E') = \{\$\}$

Para $T \rightarrow F\ T'$ tem-se $FIRST(F\ T') = \{\neg, id\}$

Para $T' \rightarrow \&\ F\ T'$ tem-se $FIRST(\&\ F\ T') = \{\&\}$

Para $T' \rightarrow \varepsilon$ tem-se $FOLLOW(T') = \{\vee, \$\}$

Para $F \rightarrow \neg\ F$ tem-se $FIRST(\neg\ F) = \{\neg\}$

Para $F \rightarrow id$ tem-se $FIRST(id) = \{id\}$

	id	$\vee$	&	$\neg$	$
E	$E \rightarrow T\ E'$			$E \rightarrow T\ E'$	
E'		$E' \rightarrow \vee\ T\ E'$			$E' \rightarrow \varepsilon$
T	$T \rightarrow F\ T'$			$T \rightarrow F\ T'$	
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow \&\ F\ T'$		$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow \neg\ F$	

Analisador Sintático Preditivo Sem Recursão

- Observação importante
  - Se em cada entrada da tabela existe apenas uma produção, então a gramática é LL(1) => não há ambiguidade!

- Exemplo: a gramática abaixo é ambígua para a sentença w, e pode ser interpretada de duas formas diferentes.

w = if <cond> then if <cond> then a else a

(1) <cmd> ::= if <cond> then <cmd> <pelse>

(2) <cmd> ::= a

(3) <pelse> ::= else <cmd>

(4) <pelse> ::= ε

(5) <cond> ::= b

	a	b	if	then	else	$
<cmd>	2		1			
<pelse>					3 / 4	4
<cond>		5				

Recuperação de Erros

- Na tabela de análise sintática, as lacunas representam situações de erro e devem ser usadas para chamar rotinas de recuperação.

- Pode-se alterar a tabela de análise para recuperar erros segundo dois modos distintos:
  - Modo pânico
  - Recuperação em nível de frase

Modo Pânico

- Na ocorrência de um erro, o analisador despreza símbolos da entrada até encontrar um token de sincronização.

- O conjunto de tokens de sincronização para um símbolo não-terminal A é formado pelos terminais em FOLLOW(A).

- Ao encontrar um token inesperado na sentença em análise:
  - Emitir mensagem de erro
  - Tomar uma das seguintes atitudes:
    - Se a entrada na tabela estiver vazia, ler o próximo token (significa descarte do token lido).
    - Se a entrada é sinc, desempilhar o não-terminal do topo.
    - Se o token do topo não é igual ao símbolo da entrada, desempilhar o token.

Modo Pânico

$E \rightarrow T\ E'$

$E' \rightarrow \vee\ T\ E' \mid \varepsilon$

$T \rightarrow F\ T'$

$T' \rightarrow \&\ F\ T' \mid \varepsilon$

$F \rightarrow \neg\ F \mid id$

FIRST(E) = {¬, id}

FIRST(E') = {v, ε}

FIRST(T) = {¬, id}

FIRST(T') = {&, ε}

FIRST(F) = {¬, id}

FOLLOW(E) = {$}

FOLLOW(E') = {$}

FOLLOW(T) = {v,$}

FOLLOW(T') = {v,$}

FOLLOW(F) = {&, v, $}

	id	∨	&	¬	$
E	E → T E'			E → T E'	sinc
E'		E' → ∨ T E'			E' → ε
T	T → F T'	sinc		T → F T'	sinc
T'		T' → ε	T' → & F T'		T' → ε
F	F → id	sinc	sinc	F → ¬ F	sinc

Modo Pânico

	id	∨	&	¬	$
E	E → T E'			E → T E'	sinc
E'		E' → ∨ T E'			E' → ε
T	T → F T'	sinc		T → F T'	sinc
T'		T' → ε	T' → & F T'		T' → ε
F	F → id	sinc	sinc	F → ¬ F	sinc

Pilha	Entrada	Ação
$ E	id ∨ & id $	E → T E'
$ E' T	id ∨ & id $	T → F T'
$ E' T' F	id ∨ & id $	F → id
$ E' T' id	id ∨ & id $	desempilha e lê símbolo
$ E' T'	∨ & id $	T' → ε
$ E'	∨ & id $	E' → ∨ T E'
$ E' T ∨	∨ & id $	desempilha e lê símbolo
$ E' T	& id $	descarta a entrada
$ E' T	id $	T → F T'
$ E' T' F	id $	F → id
$ E' T' id	id $	desempilha e lê símbolo
$ E' T'	$	T' → ε
$ E'	$	E' → ε
$	$	aceita

Recuperação em Nível de Frase

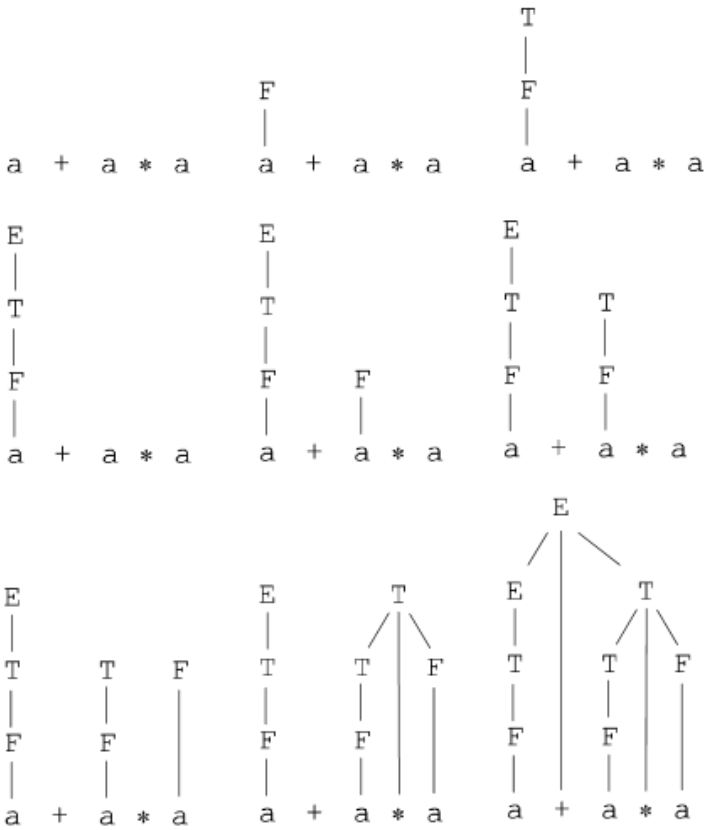
- Preencher as entradas em branco da tabela de análise sintática com apontadores para rotinas de erros

- As rotinas de tratamento de erros fazem descarte, substituição ou inserção de apenas um símbolo a cada erro descoberto.
  - É necessário ter cuidado para não provocar um loop infinito no analisador

Analisador Sintático Ascendente

- Construir a árvore de derivação sintática (ADS) para uma sentença w a partir dos símbolos de w (folhas), fazendo reduções (substituir o lado direito de uma regra pelo seu lado esquerdo) até obter o símbolo inicial S (raiz).

$$E \rightarrow E + T$$
$$E \rightarrow T$$
$$T \rightarrow T * F$$
$$T \rightarrow F$$
$$F \rightarrow (E)$$
$$F \rightarrow a$$



Analisador Sintático Ascendente - Redução

- O processo de análise sintática ascendente pode ser encarado como um processo de "reduzir" uma cadeia w para o símbolo inicial da gramática.

- Redução: operação de substituição do lado direito de uma produção pelo não-terminal correspondente do lado esquerdo
  - Para a regra $A \rightarrow \alpha$, α pode ser reduzido em A

- As principais decisões relacionadas com a análise ascendente em cada passo do reconhecimento são:
  - Como identificar a parte da cadeia w que deve ser reduzida?
  - Que produção deve ser usada na redução?

Analisador Sintático Ascendente - Redução

- Exemplo:

$$E \rightarrow E + T \mid T \qquad T \rightarrow T * F \mid F \qquad F \rightarrow (E) \mid id$$

$$w = id * id$$

- A redução segue a seguinte sequência de cadeias:

$$id * id, F * id, T * id, T * F, T, E$$

- Por definição, uma redução é o inverso de um passo em uma derivação

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * id \Rightarrow F * id \Rightarrow id * id$$

Analisador Sintático Ascendente - Redução

- O objetivo do método de análise ascendente é, portanto, construir uma derivação ao reverso

- Essa derivação é, na verdade, uma derivação mais à direita

$$E \Rightarrow T \Rightarrow T * F \Rightarrow T * id \Rightarrow F * id \Rightarrow id * id$$

- A análise ascendente ao ler a entrada, da esquerda para a direita, constrói uma derivação mais à direita ao inverso

$$id * id, F * id, T * id, T * F, T, E$$

Analisador Sintático Ascendente - Handle

- Como identificar a parte da cadeia w que deve ser reduzida?

- Informalmente, um "handle" (puxador) de uma cadeia de símbolos é uma subcadeia que casa com o corpo da produção, e cuja redução para o não-terminal do lado esquerdo representa um passo da derivação à direita ao inverso

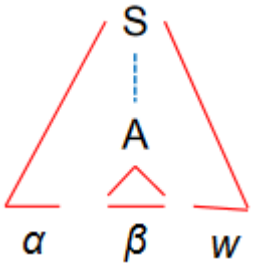
- Exemplos: handles durante a análise de $id_1 * id_2$:

$$E \rightarrow E + T \mid T \qquad T \rightarrow T * F \mid F \qquad F \rightarrow (E) \mid id$$

Forma sentencial à direita	*Handle*	Produção de redução
$id_1 * id_2$	id_1	$F \rightarrow id$
$F * id_2$	F	$T \rightarrow F$
$T * id_2$	id_2	$F \rightarrow id$
$T * F$	$T * F$	$E \rightarrow T * F$

Analisador Sintático Ascendente - Handle

- Formalmente, se $S \Rightarrow^* \alpha A w \Rightarrow \alpha\beta w$, então a produção $A \rightarrow \beta$ na posição seguindo α é um handle de $\alpha\beta w$



- Se uma gramática não for ambígua, então cada forma sentencial à direita da gramática tem exatamente um handle

Analisador Sintático Ascendente

- Analisador sintático Shift-Reduce (Empilha-Reduz):
  - É uma forma de análise ascendente em que uma pilha contém símbolos da gramática e um buffer de entrada que contém o restante da cadeia a ser reconhecida sintaticamente
  - O handle sempre aparece no topo da pilha, imediatamente antes de ser identificado
  - Usa-se o símbolo delimitador $ para marcar o fim da pilha e também o extremo direito da entrada

Analisador Sintático Ascendente

- Analisador sintático Shift-Reduce (Empilha-Reduz):
  - Mostraremos o topo da pilha à direita, em vez de à esquerda, como fizemos para a análise descendente
  - Inicialmente, a pilha está vazia, e a cadeia w representa a entrada, da seguinte forma:

Pilha	Cadeia
$	w$

Analisador Sintático Ascendente

- Analisador sintático Shift-Reduce (Empilha-Reduz):
  - Durante a análise da cadeia, o analisador sintático transfere zero ou mais símbolos da entrada para a pilha, até que uma cadeia β de símbolos da gramática presente no topo possa ser reduzida para o lado esquerdo da produção apropriada
  - O analisador repete esse ciclo até detectar um erro na entrada ou até que a pilha contenha apenas o símbolo inicial da gramática e a entrada esteja vazia:

Pilha	Cadeia
$S	$

Analisador Sintático Ascendente

- Analisador sintático Shift-Reduce (Empilha-Reduz):

Embora as principais operações sejam shift e reduce, existem quatro operações possíveis que podem ser realizadas:

- Shift (Empilha): transfere o próximo símbolo da entrada para o topo da pilha
- Reduce (Reduz): O extremo direito da cadeia a ser reduzida deve estar no topo da pilha. Localize o extremo esquerdo da cadeia no interior da pilha e decida por qual não-terminal esta cadeia será substituída
- Aceita: anuncia o término bem sucedido da análise
- Erro: a cadeia de entrada não é reconhecida
  - Chama rotina de tratamento de erros

Analisador Sintático Ascendente

- Analisador sintático Shift-Reduce (Empilha-Reduz):
  - Processo de reconhecimento da sentença [a;a]:

Pilha	Cadeia	Regra
$	[a;a]$	Empilha [
$[	a;a]$	Empilha a
$[a	;a]$	Reduz S → a
$[S	;a]$	Reduz L → S
$[L	;a]$	Empilha ;
$[L;	a]$	Empilha a
$[L;a	]$	Reduz S → a
$[L;S	]$	Reduz L → L;S
$[L	]$	Empilha]
$[L]	$	Reduz S → [L]
$S	$	Aceita

$S \rightarrow [L] \mid a$
$L \rightarrow L;S \mid S$

Analisador Sintático Ascendente

- Analisador sintático Shift-Reduce (Empilha-Reduz):

$S \rightarrow [L] \mid a$

$L \rightarrow L;S \mid S$

- Reconhecer a cadeia: [a;a]
- Haveria outras possibilidades de handles?
  - $L \rightarrow S$
  - $[L;L => ERRO
    - Cadeia não é reconhecida

Pilha	Cadeia	Ação
$	[a;a]$	Empilha [
$[	a;a]$	Empilha a
$[a	;a]$	Reduz $S \rightarrow a$
$[S	;a]$	Reduz $L \rightarrow S$
$[L	;a]$	Empilha ;
$[L;	a]$	Empilha a
$[L;a	]$	Reduz $S \rightarrow a$
$[L;S	]$	Reduz $L \rightarrow L;S$
$[L	]$	Empilha]
$[L]	$	Reduz $S \rightarrow [L]$
$S	$	Aceita

Analisador Sintático Ascendente

- Analisador sintático Shift-Reduce (Empilha-Reduz):
  - Exemplo: processo de reconhecimento da sentença id * id:

Pilha	Cadeia	Regra
$	id * id$	Empilha id
$id	* id$	Reduz F → id
$F	* id$	Reduz T → F
$T	* id$	Empilha *
$T*	id$	Empilha id
$T*id	$	Reduz F → id
$T*F	$	Reduz T → T * F
$T	$	Reduz E → T
$E	$	Aceita

E → E + T | T
T → T * F | F
F → (E) | id

Analisador Sintático Ascendente

- Conflitos durante execução do analisador shift-reduce:
  - Quando o analisador não consegue decidir se deve realizar um shift ou um reduce, temos um conflito shift/reduce
  - Quando o analisador não consegue determinar qual redução fazer, temos um conflito reduce/reduce

Analisador Sintático Ascendente

- Exemplo:
  - Dada a gramática:

S> if E then S | if E then S else S | other

  - E a configuração:

PILHA	ENTRADA
...if E then S	else .... $

  - Qual tipo de conflito é identificado durante a análise?

Analisador Sintático Ascendente

- Exemplo:
  - Dada a gramática:

    s> id (parametos) | e := e
    parametros> parametros, parametro | parametro
    parametro> id
    e> id (lista_e) | id

  - E a configuração:

	PILHA	ENTRADA
	... id (id	, id) ...

  - Qual tipo de conflito é identificado durante a análise?

Analisador Sintático Ascendente

- Duas classes de analisadores:
  - Analisadores de precedência de operadores
  - Analisadores LR (Left to right with Right-most derivation)

Analisadores LR

- Analisadores sintáticos LR(k):
  - L, verificação da entrada da esquerda para direita
  - R, constrói a derivação reversa mais a direita
  - k, número de símbolos da entrada que são usados para tomar decisão
- SLR (Simple LR): método para construção de analisadores sintáticos shift-reduce
- Analisadores LR utilizam uma tabela para tomar decisões da análise sintática

Analisadores LR

- Analisadores LR podem ser utilizados para reconhecer uma grande quantidade de construções de LP

- O método utilizado por um analisador LR é o mais conhecido método que não utiliza backtracking para shiftreduce

- Principal desvantagem: muito esforço para construir um analisador LR "do zero"

Analisadores LR

- Como um analisador shiftreduce sabe quando realizar o shift ou o reduce?

- Um analisador LR toma decisões de shiftreduce através de estados em cada "momento" da análise

  - Estados representam conjuntos de itens

- Um item de uma gramática G é uma produção de G com um ponto em alguma posição no corpo da produção

  - Exemplo: a produção A>XYZ possui quatro itens:

    A>.XYZ, A>X.YZ, A>XY.Z e A>XYZ.

    OBS: um item indica "quanto" já foi analisado de uma produção

Analisadores LR

- Uma coleção de itens LR é chamado de coleção canônica LR
- Essa coleção é a base para a construção de um AFD que é usado nas decisões de análise sintática
- Cada estado do autômato representa um conjunto de itens
- Para construir a coleção de itens LR para uma gramática, devese definir uma gramática estendida e duas funções: FECHO (closure) e GOTO
  - Se G é uma gramática com símbolo inicial S, então G‘ é a gramática estendida tal que S'>S, é o símbolo inicial de G‘
  - A aceitação de uma entrada ocorre apenas quando o analisador reduz de S para S'

Analisadores LR

- FECHO de conjuntos de itens:
  - Se I é um conjunto de itens para uma gramática G, então FECHO(I) é o conjunto de itens construídos de I com as regras:
    1. Inicialmente, adicionar todo item em I para FECHO(I)
    2. Se A>a.Bb está em FECHO(I) e B>y é uma produção, então adicionar o item B>.y para FECHO(I), se já não está I á .

       OBS: Aplicar a regra 2 até não existirem itens que podem ser adicionados para o FECHO(I)
  - A>a.Bb no FECHO(I) indica que, em algum momento, esperase obter uma substring derivada de Bb na entrada
    - A substring derivada de Bb terá um prefixo derivado de B. Logo, adicionase itens para as produções de B. Ou seja, se B>y, então, B>.y está em FECHO(I)

Analisadores LR

- A função FECHO pode ser computada conforme o algoritmo a seguir:

```
SetOfItems FECHO(I){
    J=I;
    repeat
      for (cada item A->a.Bb em J)
        for (cada produção B->y de G)
          if (B->.y não está em J)
            add B->.y em J
    until não existem mais itens para adicionar em J
    return J;
}
```

- Note que se uma produção B é adicionada para o Fecho(I) com o ponto no início da produção, então todas as produções de B serão adicionadas para o fecho

Analisadores LR

- Exemplo: considere a gramática:

E'> E

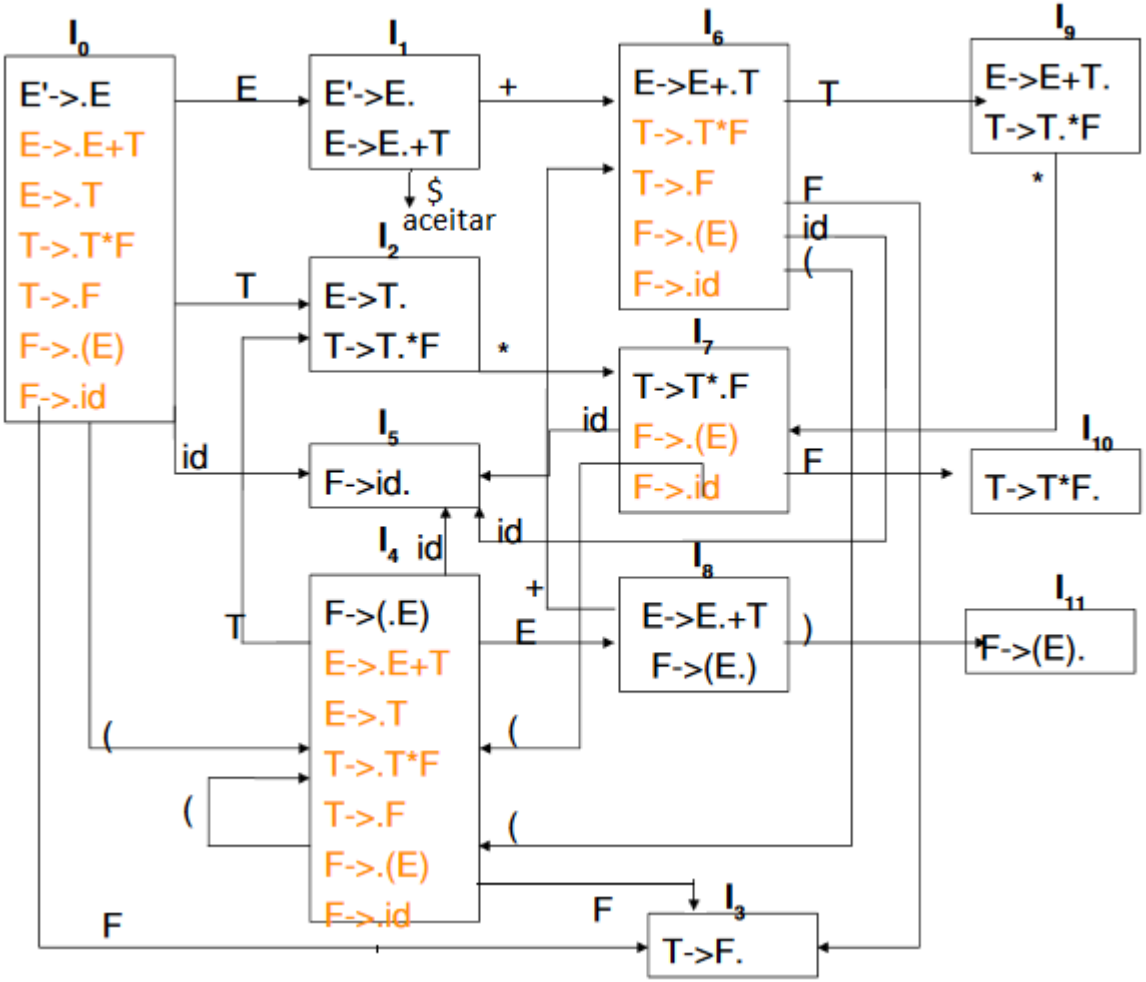
E > E + T | T

T > T * F | F

F > (E) | id

Se I é o conjunto com um item {E'>.E}, então, FECHO(I) contém o conjunto de itens I_0 a seguir:

Analisadores LR



Analisadores LR

- Os conjuntos de itens de interesse são divididos em:
  - Itens de kernel: o item inicial (S'>S) e todos os itens cujos pontos não estão no início da regra de produção
  - Itens nãokernels: todos os itens com pontos no início da regra de produção, exceto o item inicial
- Cada conjunto de itens de interesse é formado pelo FECHO de um conjunto de itens de kernel
  - Itens nãokernels são representados na cor laranja, na figura anterior

Analisadores LR

- A função GOTO possibilita definir as transições no autômato LR
- Os estados do autômato correspondem ao conjunto de itens GOTO(I,X) indica a transição de I sob a entrada X
  - I é um conjunto de itens e X é um símbolo da gramática
- A função GOTO(I,X) é definida como o fecho do conjunto de todos os itens A>aX.b tal que A>a.Xb está em I
- Exemplo: verifique novamente a figura do autômato LR e, dado o conjunto de itens {E'>E., E>E.+T}, então, o GOTO(I,+) contém:

E->E+.T
T->.T*F
T->.F
F->.(E)
F->.id

Para computar Goto(I,+), examina-se I nos itens que possuem + imediatamente à direita do ponto. Move-se então o ponto após o + e calcula-se o fecho desse item.

Analisadores LR

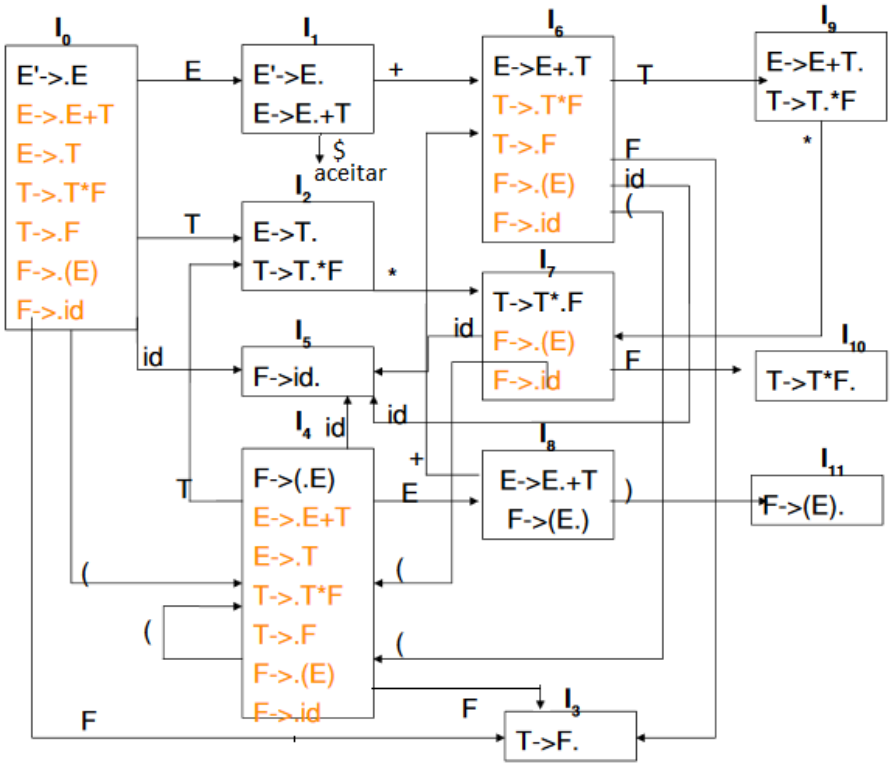
- Algoritmo para computar a coleção canônica de itens de LR para uma gramática G'

```
void items(G'){
   C=Fecho({S'->.S}); //Observe que S'->S é o símbolo inicial de G'
    repeat
      for (cada conjunto de itens I em C)
        for (cada símbolo da gramática X)
          if (Goto(I,X) não está vazio e não está em C)
            add Goto(I,X) em C
    until não exista conjuntos para serem adicionados  em C
}
```

Analisadores LR

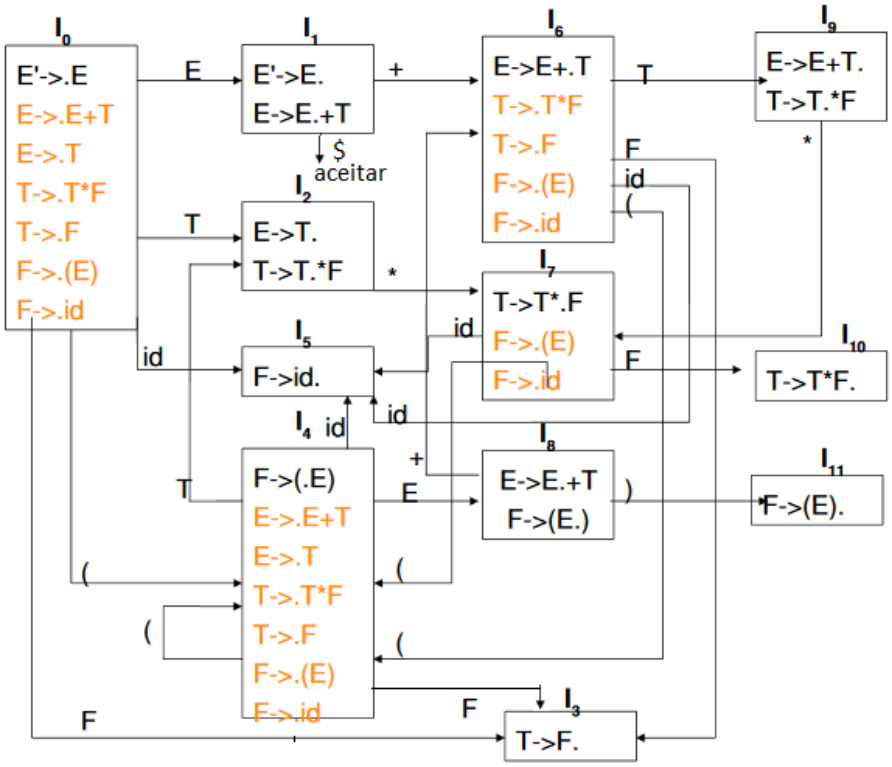
- A ideia por trás do analisador é realizar a análise a partir do autômato LR(0).
  - Nesse autômato, os estados são conjuntos de itens da coleção canônica e as transições são dadas pela função GOTO

- O estado inicial do autômato LR é dado pelo fecho do estado inicial de G'. Um estado j referese ao estado correspondendo ao conjunto de itens I_j



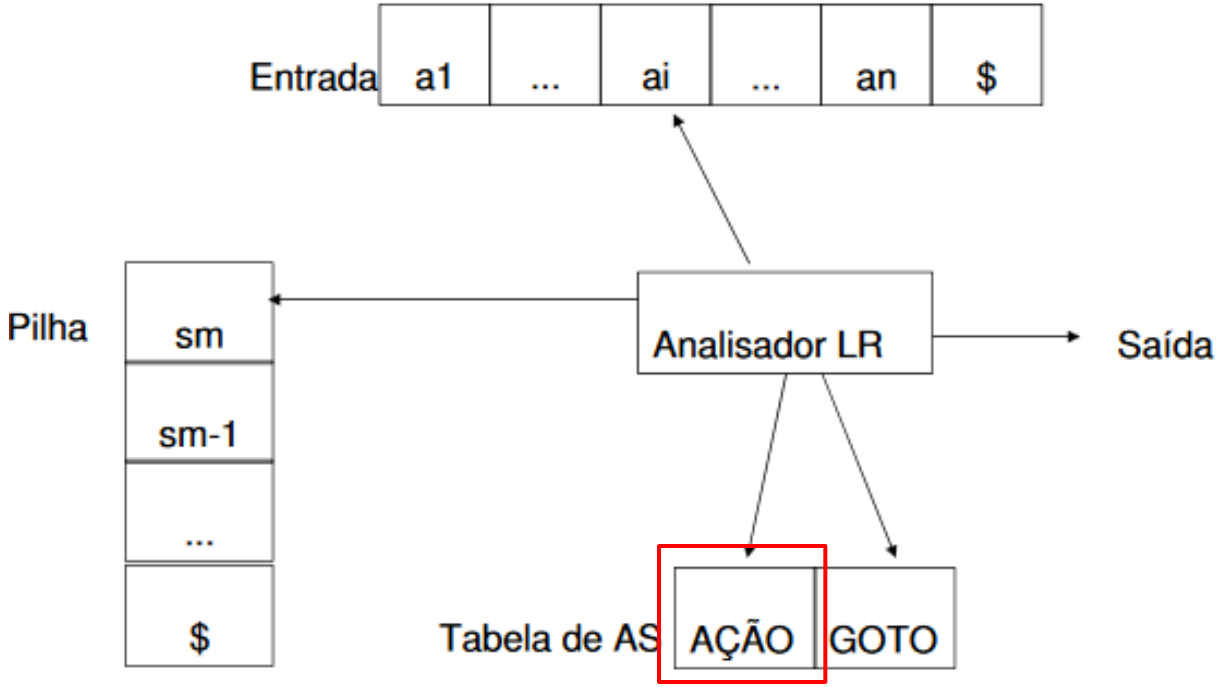
Analisadores LR

- As decisões de shiftreduce são feitas da seguinte maneira:
  - Supor que uma string w de símbolos de G' inicia o autômato LR com uma transição do estado inicial 0 para o estado j
  - Então, deslocase (shift) o próximo símbolo da entrada a se j tem uma transição com a
  - Se não existe tal transição, fazse a redução (reduce). Os itens que compõem j indicam qual produção usar na redução.



Analisadores LR

- Como utilizar o autômato para realizar a análise sintática LR?
  - Construir uma tabela de análise sintática de acordo com o seguinte algoritmo



Analisadores LR

- A função AÇÃO recebe como argumento um estado *i* e um terminal *a*. O retorno de AÇÃO[i,a] pode ser:
  1. Empilhar j, onde j é um estado. Desloca *a* para a pilha. Use estado j para representar *a*
  2. Reduz A>b. Reduz b, no topo da pilha, para A
  3. Aceitar. Analisador aceita a entrada e finaliza
  4. Erro. Dependendo do erro, pode finalizar ou executar alguma ação

- OBS: GOTO[I_i,A]=I_j, então GOTO mapeia o estado i e um nãoterminal A para o estado j

Analisadores LR – Tabela SRL

- As entradas AÇÃO e GOTO são construídas a partir do algoritmo
- ENTRADA: gramática G'

1. Construir $C=\{I_0, I_1, ..., I_n\}$, a coleção de itens LR para G'

2. Estado *i* é construído de I_i. As ações para o estado *i* são determinadas como segue:

a) Se [A->c.ab] está em I_i e $GOTO(I_i,a)=I_j$, então determinar AÇÃO[i,a]= shift j. a deve ser um terminal

b) Se [A->B.] está em I_i, então AÇÃO[i,a]=reduce A->B, para todo a em FOLLOW(A); A não deve ser S'.

c) Se [S'->S.] está em I_i, então AÇÃO[i,$]=accept

3. As transições GOTO são construídas a partir dos não-terminais usando a regra: if $GOTO(I_i,A)=I_j$, então GOTO[i,A]=j

4. Todas as entradas não definidas através dos passos 2) e 3) indicam "erro"

5. O estado inicial do parser é aquele do conjunto de itens [S'->S]

Analisadores LR

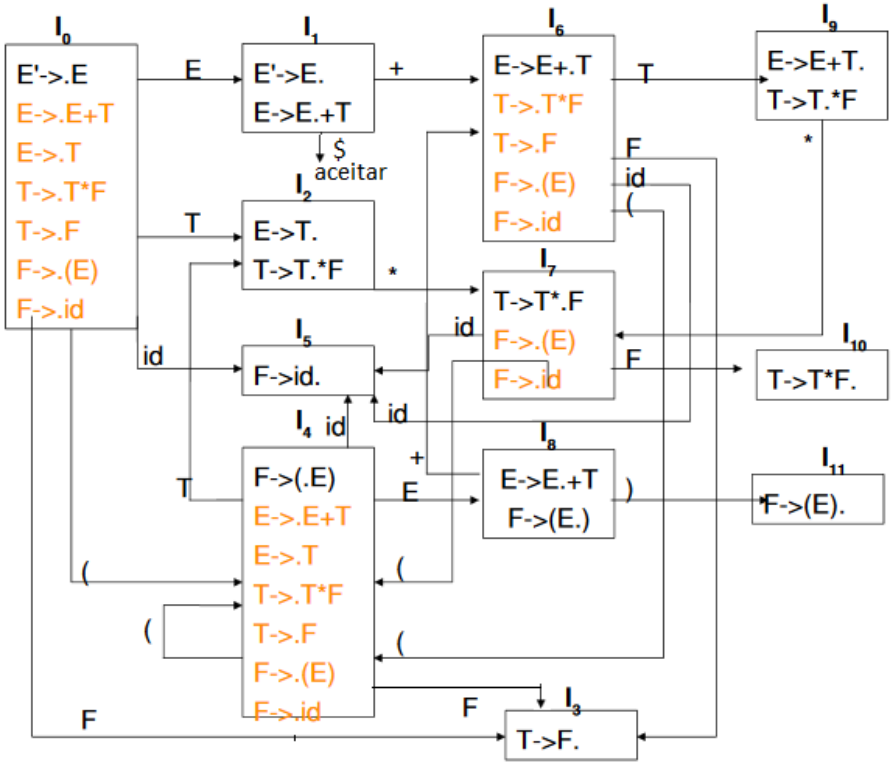
- Considere as regras de produção de G:

1) E> E+T; 2) E> T; 3) T> T*F;
4) T> F; 5) F> (E); 6) F>id

- Os códigos para as ações são:
  - si: significa shift e empilhar estado i
  - rj: significa reduce para a produção j
  - acc: significa accept
  - Espaço em branco significa erro

Analisadores LR

Estado	AÇÃO						GOTO		
	ID	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5		s4					9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			



Analisadores LR

- Exemplo: construa o autômato LR(0) e a tabela de análise SLR para a gramática a seguir:

E -> T + E

E -> T

T -> x

Analisadores LR

- Exercício: construa o autômato LR(0) e a tabela de análise SLR para a gramática a seguir:

S -> (L)

S -> x

L -> S

L -> L,S

Analisadores LR

- O comportamento do analisador sintático é então definido da seguinte maneira:
  - Lê a, o símbolo atual da entrada, e s, o estado atual (no topo da pilha) e consulta a entrada AÇÃO[s,a] na Tabela de Análise Sintática.
  - O retorno de AÇÃO[s,a] deve ser um dos quatro possíveis: deslocar (shift) para estado s, reduzir, aceitar ou erro.

Analisadores LR – Tabela SLR

- ENTRADA: string w e tabela com funções AÇAO e GOTO para gramática G
- SAÍDA: se w está em L(G), aceitação para w, senão, erro!

```
Seja a, o primeiro símbolo de w$
while (1){
   Seja s o estado no topo da pilha
   if (AÇÃO[s,a]==shift t){
      push t;
      Seja a o próximo símbolo da entrada;
   }else if (AÇÃO[s,a]==reduce A->b){
       pop |b| símbolos da pilha;
       Seja t o estado no topo da pilha;
       push GOTO[t,A];
   }else if (AÇÃO[s,a]==aceitar) break;
   else erro();
}
```

Analisadores LR

- Exemplo: considere a gramática anterior e a entrada id*id. Faça a análise desta entrada utilizando a tabela SLR abaixo.

Pilha	Símbolos	Entrada	Ação
0	$	id*id$	shift para 5
0 5	$id	*id$	reduce para F->id
0 3	$F	*id$	reduce para T->F
0 2	$T	*id$	shift para 7
0 2 7	$T*	id$	shift para 5
0 2 7 5	$T*id	$	reduce para F->id
0 2 7 10	$T*F	$	reduce para T->T*F
0 2	$T	$	reduce para E->T
0 1	$E	$	accept

Estado	AÇÃO						GOTO		
	ID	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5		s4					9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

1) E> E+T; 2) E> T; 3) T> T*F;

4) T> F; 5) F> (E); 6) F>id

Analisadores LR

- Exercício: considere a gramática anterior e a entrada id*id + id. Faça a análise desta entrada utilizando a tabela SLR abaixo.

Pilha	Símbolos	Entrada	Ação
0	$	id*id +id$	

Estado	AÇÃO						GOTO		
	ID	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5		s4					9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

1) E> E+T; 2) E> T; 3) T> T*F;

4) T> F; 5) F> (E); 6) F>id

Analisadores LR

- Nem todas as gramáticas podem ser analisadas por métodos SLR. Considere a seguinte gramática como exemplo:

S-> L = R | R

L ->*R | id

R -> L

Analisadores LR

- A coleção canônica de conjuntos de itens L(0) para a gramática é:

  S-> L = R | R

  L ->*R | id

  R -> L

- Observe o conflito em:
  - AÇÃO[2,=]
    - Primeiro item de I_2: s6
    - Segundo item de I_2 : reduz
    - R -> L (ver FOLLOW(R))

I_0:
S´ ->.S
S -> .L=R
S -> .R
L -> .*R
L -> .id
R -> .L

I_1:
S´ -> S.

I_2:
S -> L.=R
R -> L.

I_3:
S -> R.

I_4:
L -> *.R
R -> .L
L -> .*R
L -> .id

I_5:
L -> id.

I_6:
S -> L=.R
R -> .L
L -> .*R
L -> .id

I_7:
L -> *R.

I_8:
R -> L.

I_9:
S -> L=R.

Analisadores LR

- Analisadores SLR não são poderosos o suficiente para lembrar o contexto a esquerda para decidir que ação deve tomar.
  - Solução: incorporar nos itens o primeiro símbolo da entrada ainda não lido
    - Existem dois métodos LR com esta característica: LR canônico e LALR
      - Estes não resolvem conflitos para qualquer gramática arbitrária

Analisadores LR

- LR canônico, ou só LR: usa uma tabela construída a partir do conjunto de itens denominados itens LR(1)

- Item LR(1) é da forma [A $\rightarrow \alpha.\beta$, T]
  - A $\rightarrow \alpha\beta$ é uma produção
  - O ponto em A $\rightarrow \alpha.\beta$ denota a posição
  - T é um símbolo terminal ou o marcador de término ($)

- Item [A $\rightarrow \alpha.\beta$, T] significa
  - O parser já analisou α
  - Se analisa β e o próximo símbolo é T então o parser deve reduzir com A $\rightarrow \alpha\beta$

- Consequência?
  - Modificar: FECHO, GOTO e tabela AS

Analisadores LR

- Funções para gerar LR(1)

```
SetOfItems FECHO(I){
  repeat
    for (cada item [A->C.BD,a] em I)
      for (cada produção B->y em G')
        for (cada terminal b em FIRST(Da))
          add [B->.y,b] para I;
  até que itens não sejam mais adicionados para I;
  return I;
}
```

```
SetOfItems GOTO(I,X){
  iniciar J como conjunto vazio;
  for (cada item [A->C.XD,a] em I)
    add item [A-> CX.D,a] para J;
  return FECHO(J);
}
```

```
void items(G'){
  iniciar C=FECHO({[S'->.S,$]});
  repeat
    for (cada conjunto I em C)
      for (cada símbolo X da gramática)
        if (GOTO(I,X)!=vazio e não está em C)
          add GOTO(I,X) para C
  até que nenhum novo conjunto seja adicionado para C
}
```

Analisadores LR

- Considere a seguinte gramática estendida:

$$S' \rightarrow S$$
$$S \rightarrow CC$$
$$C \rightarrow cC \mid d$$

- Após aplicar o algoritmo FECHO para I={[S'>.S,$]}, temos o seguinte conjunto I_0

I_0

S'-> .S, $
S->.CC,$
C->.cC,c/d
C->.d, c/d

Analisadores LR

- Como gerar o conjunto de itens:
  - Iniciamos com o fechamento de {[S'>.S,$]} no estado I_0
    - Temos: [A $\rightarrow \alpha$. Bβ,a] - função FECHO
      - A = S´, $\alpha = \varepsilon$, B = S, $\beta = \varepsilon$ e a = $
    - Adicionar [B -> .y,b] para cada produção B -> y e terminal b em FIRST(βa)
      - B -> y deve ser S -> CC. Como β é ε e a = $ => FIRST(βa) = $
    - Até aqui, temos:

I_0

S'-> .S, $
S->.CC,$

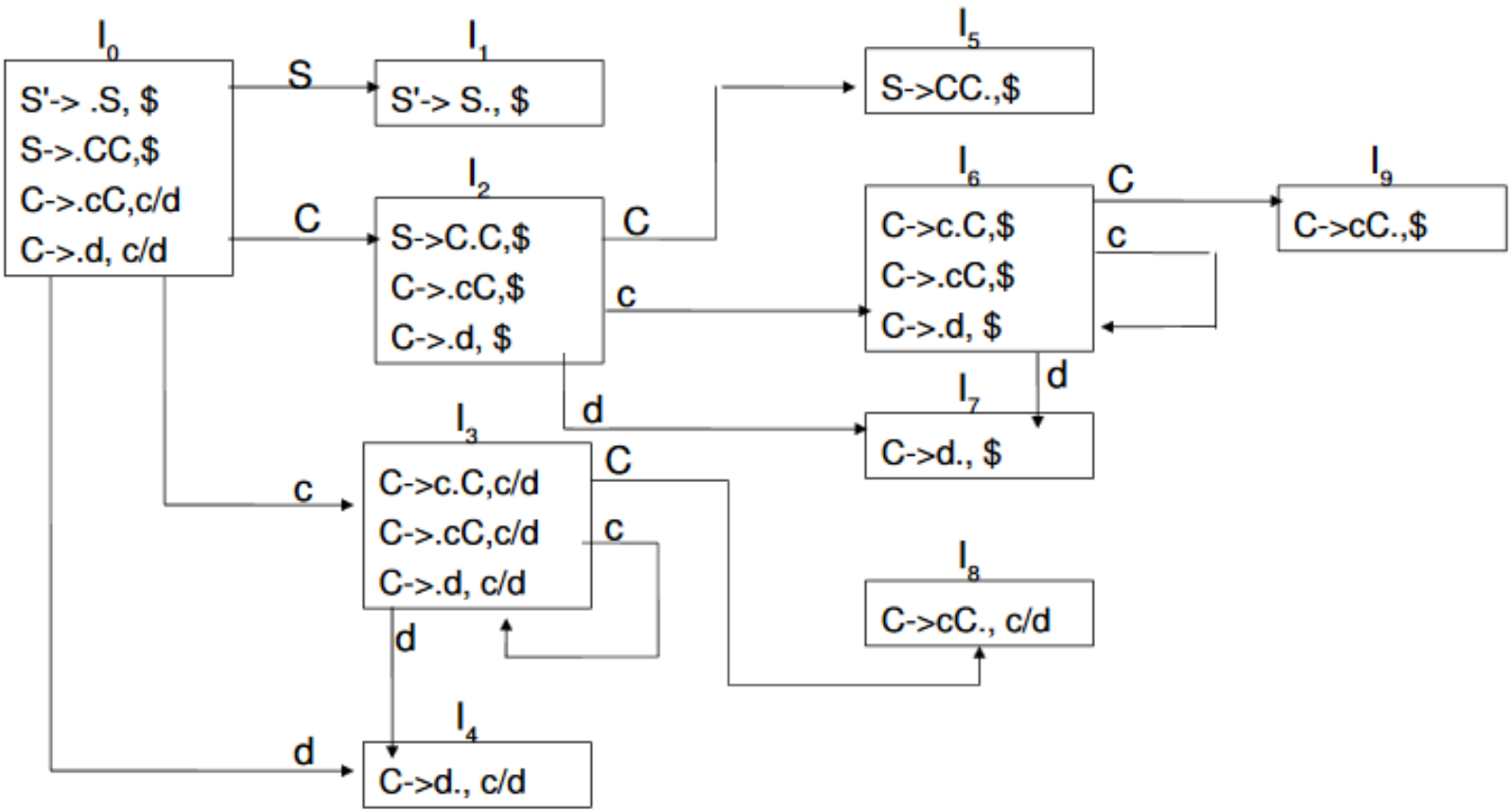
    - Depois, calcular o fechamento sobre todos os itens de [C -> .y,b]
      - Para [S ->.CC,$], temos: A = S, $\alpha = \varepsilon$, B = C, β = C e a = $
      - Lembrar que b em FIRST(βa) = {c,d}
      - Adicionar: [C -> .cC,c], [C -> .cC,d], [C -> .d,c] e [C -> .d,d]
      - No final obtemos o conjunto de itens ao lado

I_0

S'-> .S, $
S->.CC,$
C->.cC,c/d
C->.d, c/d

Analisadores LR

- Grafo GOTO para a gramática do slide 170



Analisadores LR – Tabela LR Canônica

- As entradas AÇÃO e GOTO são construídas a partir do algoritmo
- ENTRADA: gramática G'

1. Construir C={I_0, I_1, ..., I_n}, a coleção de itens LR(1) para G'

2. Estado *i* é construído de I_i. As ações para o estado *i* são determinadas como segue:

a) Se [A->C.aD,b] está em I_i e GOTO(I_i,a)=I_j, então determinar AÇÃO[i,a]= shift j. a deve ser um terminal

b) Se [A->C.,a] está em I_i, A!=S' então AÇÃO[i,a]=reduce A->C.

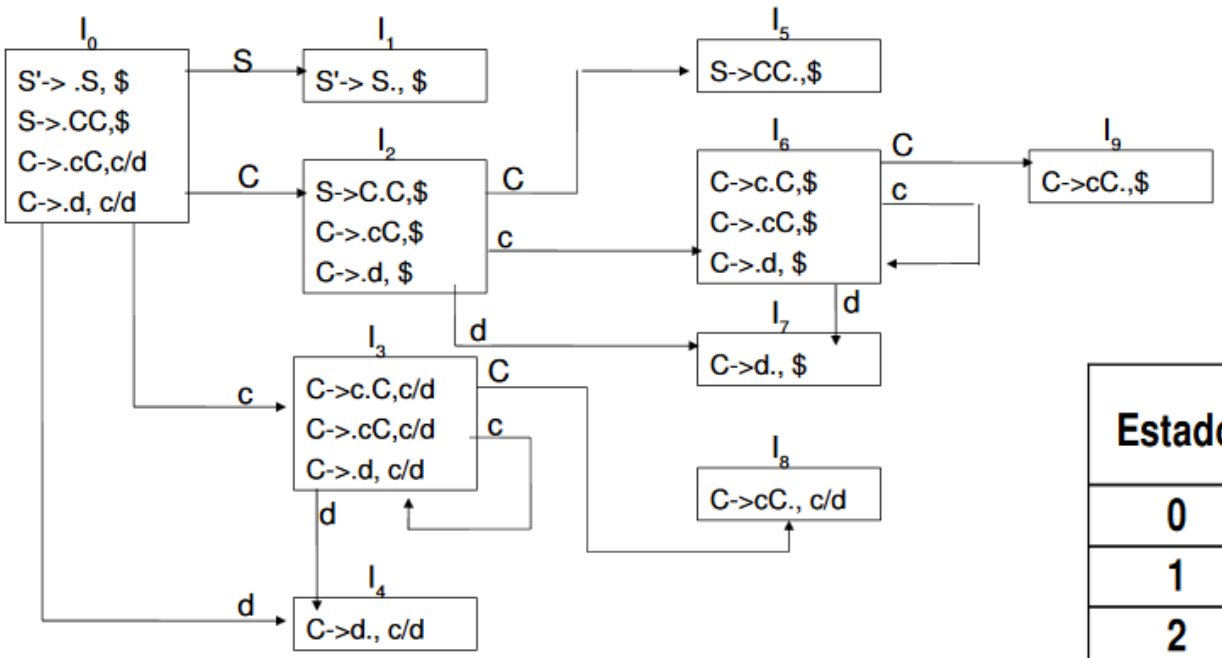
c) Se [S'->S.,$] está em I_i, então AÇÃO[i,$]=accept

3. As transições GOTO são construídas a partir dos não-terminais usando a regra: if GOTO(I_i,A)=I_j, então GOTO[i,A]=j

4. Todas as entradas não definidas através dos passos 2) e 3) indicam "erro"

5. O estado inicial do parser é aquele do conjunto de itens [S'->.S,$]

Analisadores LR



S'->S, 1) S-> CC, 2) C-> cC, 3) C-> d

Estado	AÇÃO			GOTO	
	c	d	$	S	C
0	s3	s4		1	2
1			acc		
2	s6	s7			5
3	s3	s4			8
4	r3	r3			
5			r1		
6	s6	s7			9
7			r3		
8	r2	r2			
9			r2		

Analisadores LR

- Exercício: Construir o autômato LR(1) e a tabela LR canônica para a gramática a seguir.

S-> L = R | R

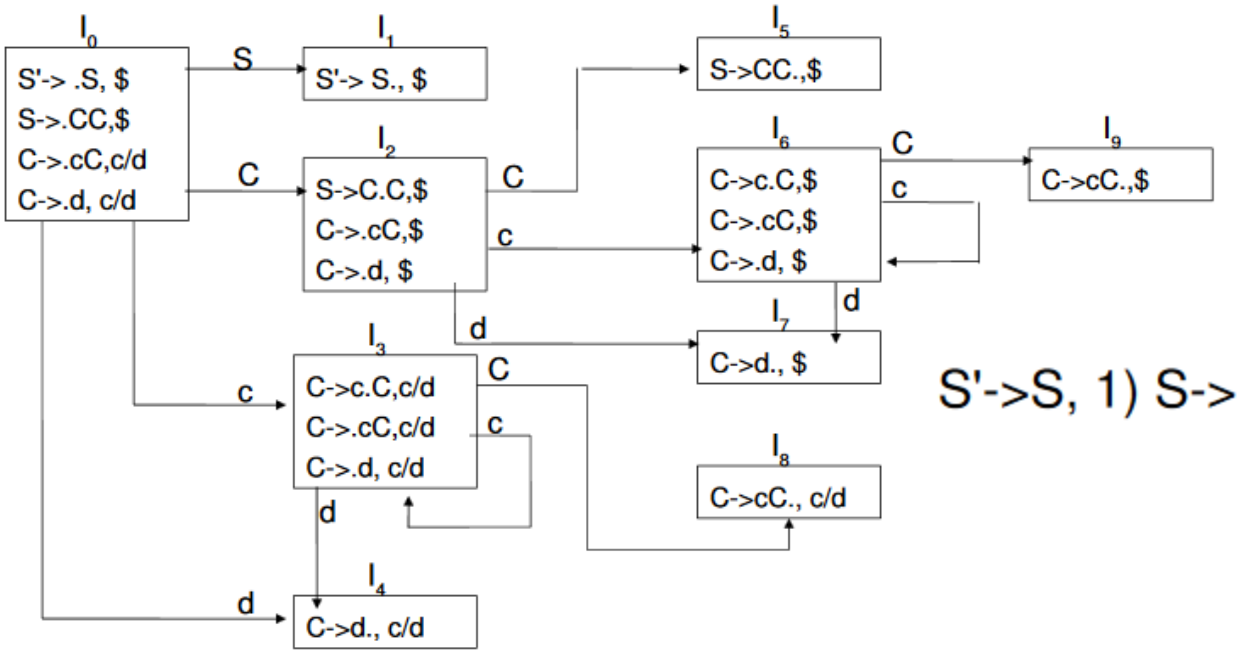
L ->*R | id

R -> L

Analisadores LR - LARL (Look Ahead LR)

- Incorpora símbolos lookahread nos itens LR(0)
- Tabelas obtidas por este método são menores que as tabelas de um analisador LR canônico, além de conseguir representar muitas construções de LP
- Considerando a gramática LR(1) anterior, observe que os estados 4 e 7 têm itens cujo primeiro componente (o item LR(0)) é igual (Ex: C>d.)
- Dessa forma, podese considerar que esses estados possuem itens com o mesmo core e, portanto, podem ser unidos para minimizar espaços nas tabelas AÇÃO e GOTO
- Existem outros conjuntos de itens que podem ser unidos?

Analisadores LR - LARL (Look Ahead LR)



S'->S, 1) S-> CC, 2) C-> cC, 3) C-> d

- Existem três pares de conjuntos de itens que podem ser unidos:
  - I_3 e $I_6 = I_{36}$

    C -> c.C, c/d/$
    C-> .cC, c/d/$
    C -> .d, c/d/$

  I_4 e $I_7 = I_{47}$

  C -> d., c/d/$

  I_8 e $I_9 = I_{89}$

  C -> cC., c/d/$

Analisadores LR – Tabela LALR

- ENTRADA: gramática G'
- SAÍDA: TABELA LALR

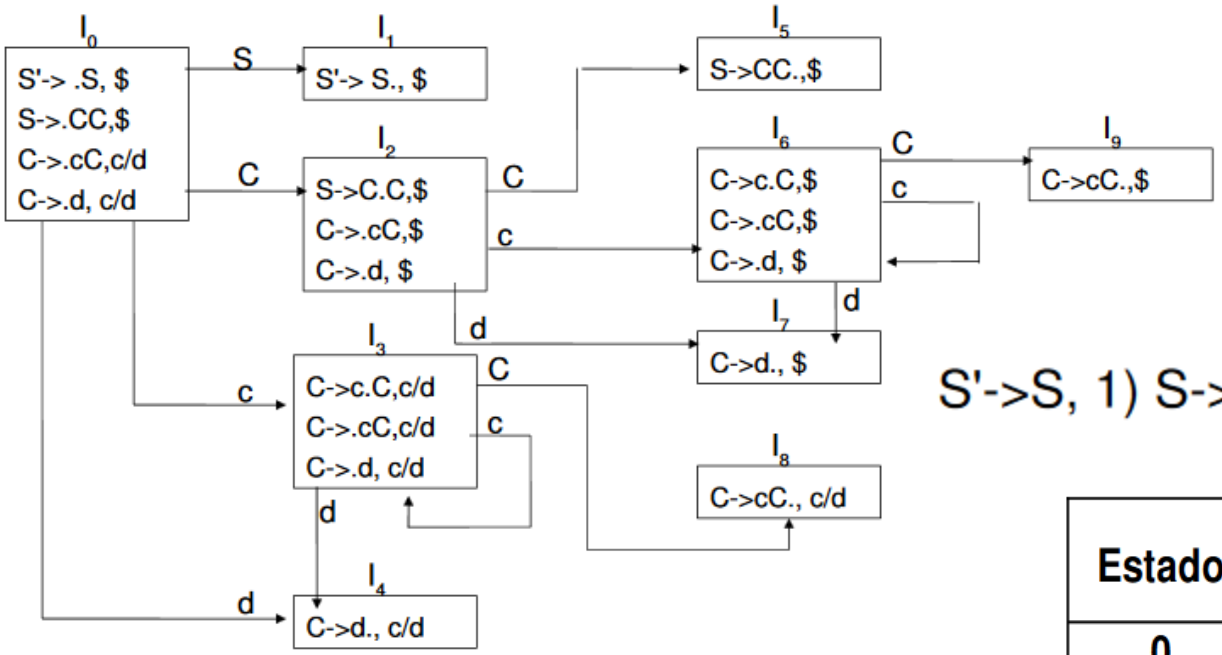
1. Construir $C=\{I_0, I_1, ..., I_n\}$, a coleção de itens LR(1)

2. Encontrar itens de LR(1) com o mesmo *core* e uni-los

3. Seja $C'=\{J_0, J_1, ..., J_m\}$ os conjuntos resultantes do passo 2. O conteúdo da tabela AÇÃO para o estado *i* é definido de J*i* da mesma forma que no algoritmo para construção da tabela LR(1). Se existir um conflito, o algoritmo falha e a gramática não é LALR(1)

4. A tabela GOTO é construída a partir de J, J=resultado da união entre os conjuntos de itens $I_1, I_2, ..., I_k$. Então, os *cores* dos $GOTO(I_1,X)$, $GOTO(I_2,X)$,..., $GOTO(I_k,X)$ são os mesmos. Seja K=GOTO(J,X)

Analisadores LR - LARL (Look Ahead LR)



S'->S, 1) S-> CC, 2) C-> cC, 3) C-> d

- I_3 e $I_6 = I_{36}$

C -> c.C, c/d/$
C-> .cC, c/d/$
C -> .d, c/d/$

I_4 e $I_7 = I_{47}$

C -> d., c/d/$

I_8 e $I_9 = I_{89}$

C -> cC., c/d/$

Estado	AÇÃO			GOTO	
	c	d	$	S	C
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

Recuperação de Erros - Analisadores LR

- Erros são detectados ao ler a tabela AÇÃO
  - Nunca ao ler a tabela GOTO

- Duas estratégias de recuperação:
  - Modo pânico
  - Recuperação em nível de frase

Modo Pânico - Analisadores LR

- Procure na pilha (de cima para baixo) um estado que tenha um GOTO para um não-terminal A
  - Ou seja, ache o momento em que A começou a ser analisado
- Joga fora todos os estados da pilha, inclusive esse encontrado
- Então basta ignorar os símbolos de entrada, até achar um que possa vir legitimamente após A
- Essa estratégia tenta eliminar a frase contendo o erro sintático. O analisador "finge" que reconheceu A corretamente, e continua a análise

Recuperação em Nível de Frase - Analisadores LR

- Examina-se cada entrada de erro na tabela e decide-se, com base na linguagem, qual erro do programador poderia gerar esse erro
  - Pode-se projetar uma rotina de recuperação
  - Cada entrada vazia da tabela pode ser um ponteiro para uma rotina
  - As ações da rotina podem incluir inserção/remoção de símbolos da pilha/entrada